第六章

树和二叉树

6.1 树的类型定义

6.2 二叉树的类型定义

6.3 二叉树的存储结构

6.4 二叉树的遍历

6.5 线索二叉树

6.6 树和森林的表示方法

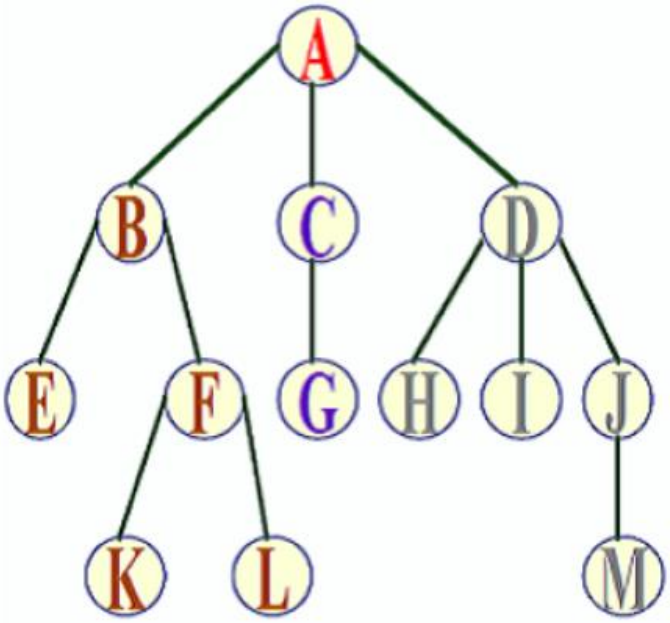
6.7 树和森林的遍历

6.8 哈夫曼树与哈夫曼编码

6.1 树的类型定义

一、树的定义

- ✓ 树是n（n≥0）个结点的有限集。
- ✓ 当n=0时称为空树；
- ✓ 在任意一棵非空树中，有且仅有一个称为根的结点，其余的结点可分为m(m ≥0)个互不相交的有限集T1, T2, …, Tm, 其中每一个集合又称为一棵树，并且称为根的子树。同理，每一棵子树又可以分为若干个互不相交的有限集…



二、抽象数据类型树的定义

ADT Tree{

数据对象 D:

D是具有相同特性的数据元素的集合。

数据关系 R:

若D为空集，则称为空树 。

否则:

(1) 在D中存在唯一的称为根的数据元素root;

(2) 当n>1时，其余结点可分为m (m>0)个互不相交的有限集$T_1, T_2, \ldots, T_m$，其中每一棵子集本身又是一棵符合本定义的树，称为根root的子树。

基本操作：

- 查 找 类
- 插 入 类
- 删 除 类

}ADT Tree

查找类：

Root(T) // 求树的根结点

Value(T, cur_e) // 求当前结点的元素值

Parent(T, cur_e) // 求当前结点的双亲结点

LeftChild(T, cur_e) // 求当前结点的最左孩子

RightSibling(T, cur_e) // 求当前结点的右兄弟

TreeEmpty(T) // 判定树是否为空树

TreeDepth(T) // 求树的深度

TraverseTree(T, Visit()) // 遍历

插入类：

InitTree(&T) // 初始化置空树

CreateTree(&T, definition)
// 按定义构造树

Assign(T, cur_e, value)
// 给当前结点赋值

InsertChild(&T, &p, i, c)
// 将以c为根的树插入为结点p的第i棵子树

删除类：

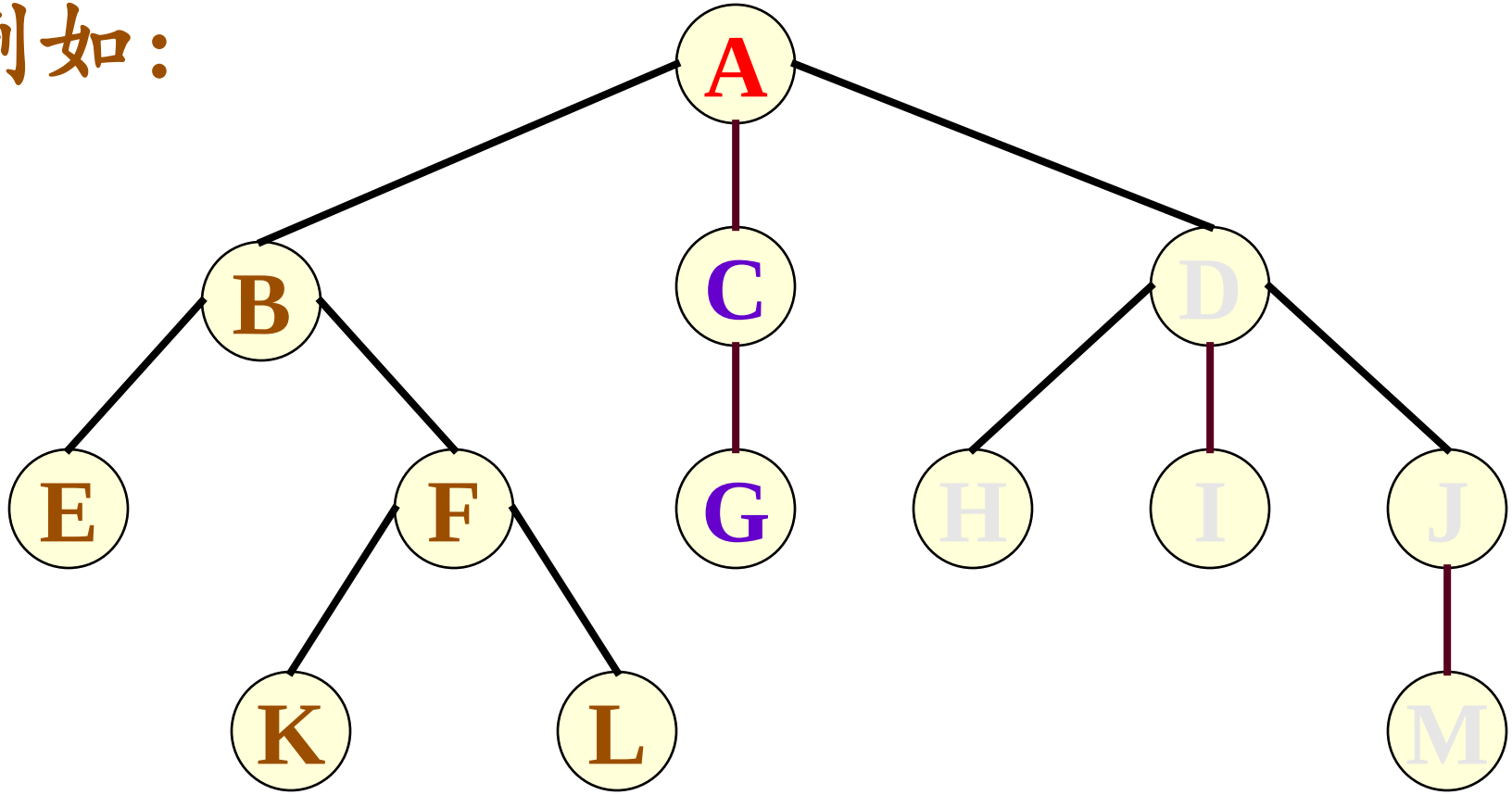
ClearTree(&T) // 将树清空

DestroyTree(&T) // 销毁树的结构

DeleteChild(&T, &p, i)

// 删除结点p的第i棵子树

例如：



A(B(E, F(K, L)), C(G), D(H, I, J(M)))

树根

T_1 T_2 T_3

有向树：

（1）有确定的根；

（2）树根和子树根之间为有向关系。

有序树：

子树之间存在确定的次序关系。

无序树：

子树之间不存在确定的次序关系。

对比树型结构和线性结构的结构特点

线性结构		树型结构
第一个数据元素 (无前驱)	→	根结点 (无前驱)
最后一个数据元素 (无后继)	→	多个叶子结点 (无后继)
其它数据元素 (一个前驱、 一个后继)	→	其它数据元素 (一个前驱、 多个后继)

三、基本术语

结点：数据元素+若干指向子树的分支

结点的度： 分支的个数

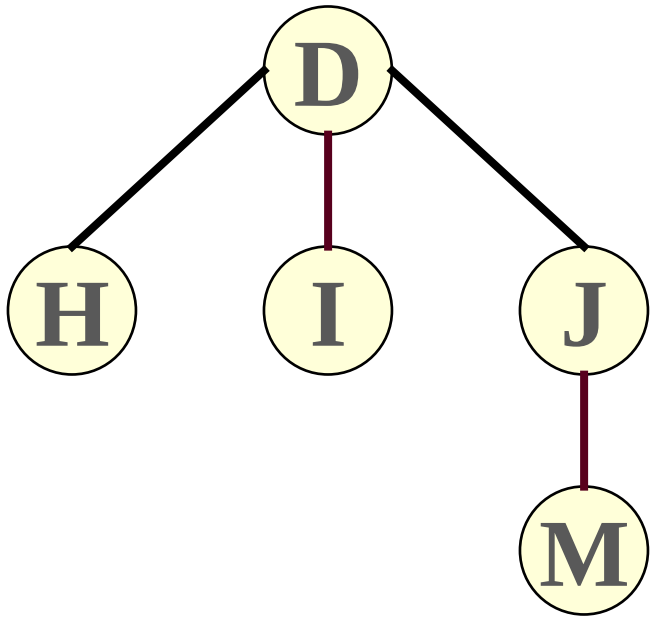
树的度：树中所有结点的度的最大值

叶子结点：度为零的结点

分支结点：度大于零的结点

(从根到结点的)**路径：**

由从根到该结点所经分支和结点构成



孩子结点：结点的子树的根称为该结点的孩子结点；

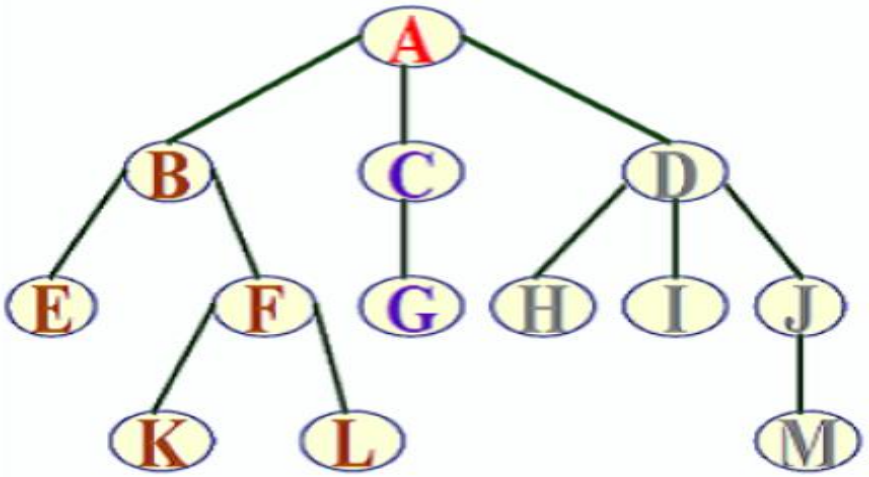
双亲结点：相应地，该结点称为孩子的双亲结点；

兄弟：具有同一父结点的子结点互称兄弟；

堂兄弟：其双亲在同一层的结点互为堂兄弟；

祖先结点：从根到该结点所经分支上的所有结点；

子孙结点：以某结点为根的子树中任一结点都称为该结点的子孙；



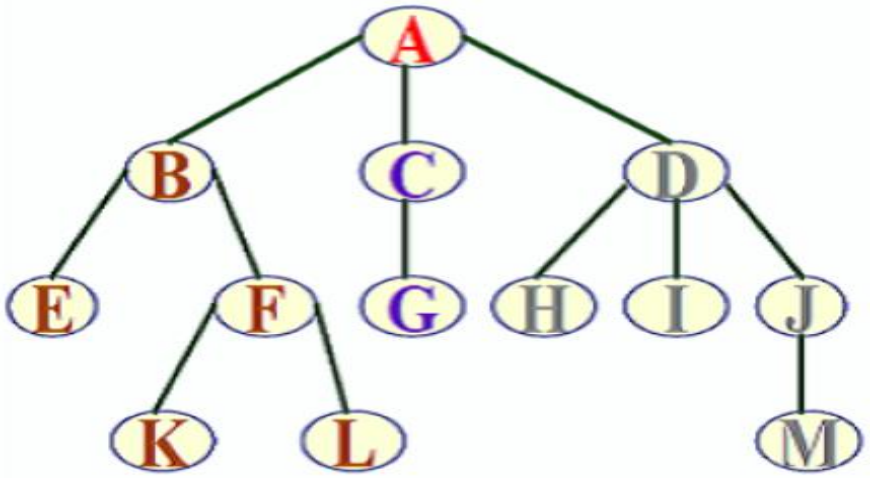
结点的层次：从根结点到该结点所经过的路径长度加1；

树的深度：树中叶子结点具有的最大层次数；

有序树：如果将树中结点的各子树看成从左至右是有次序的（即不能互换），则称该树为有序树；

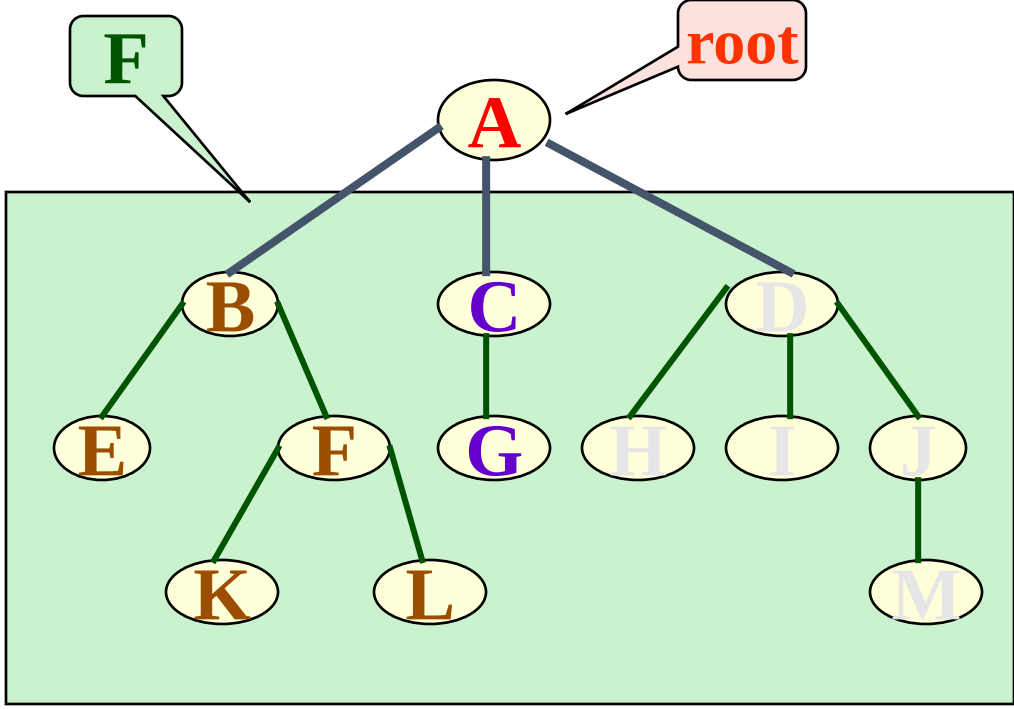
第一个孩子：在有序树中，最左边的子树的根称为第一个孩子；

最后一个孩子：最右边的子树的根称为最后一个孩子；



森林：

是m（m≥0）棵互不相交的树的集合



任何一棵非空树是一个二元组

Tree =（root，F）

其中：root 被称为根结点

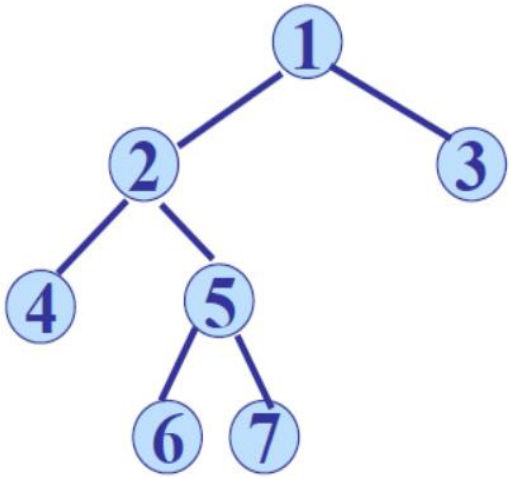
F 被称为子树森林

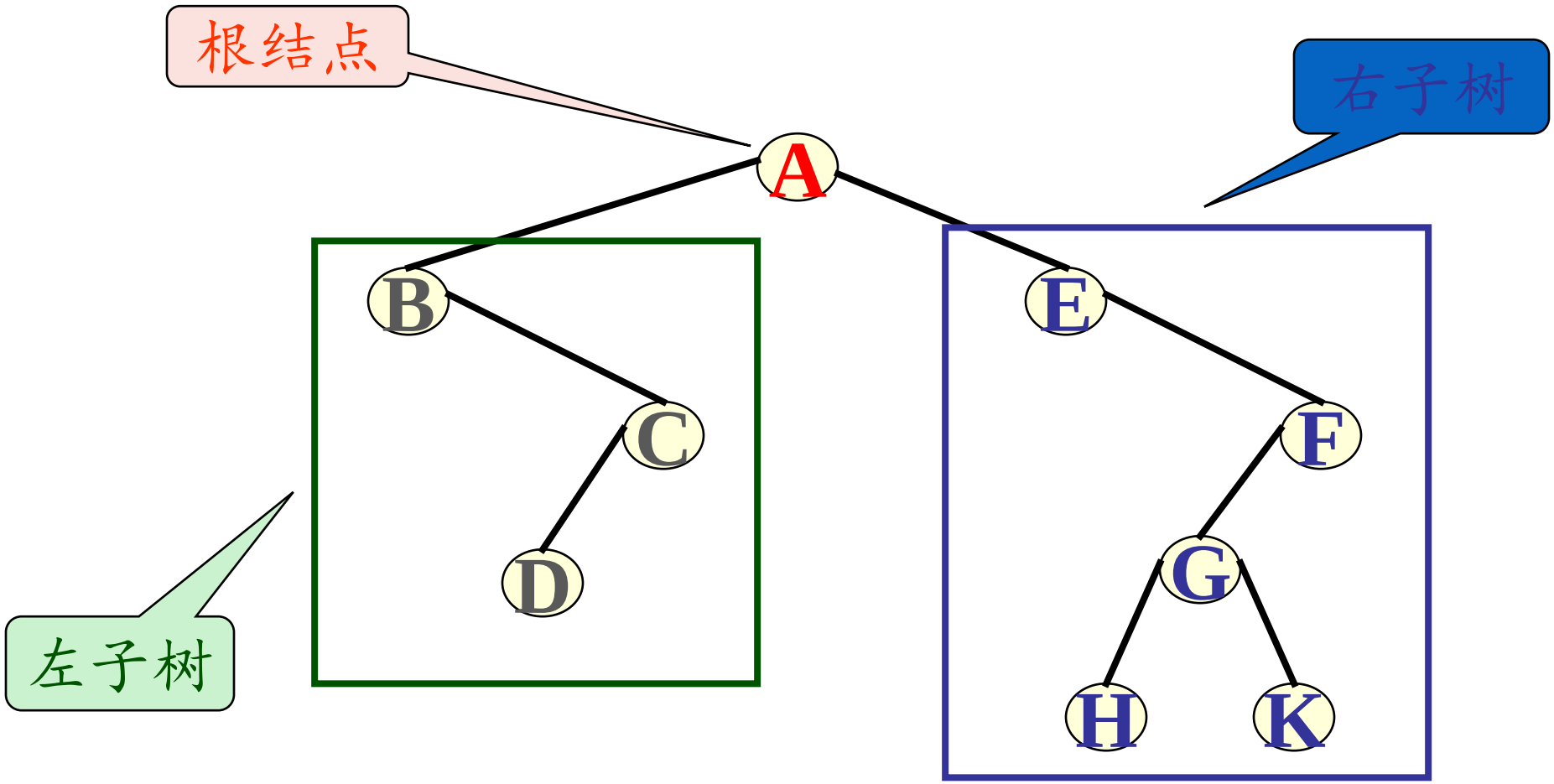
6.2 二叉树的类型定义

一、二叉树的定义

二叉树是n（n≥0）个结点的有限集合，这个集合或是空集，或是由一个根结点以及两棵互不相交的、被称为根的左子树和右子树所组成；左子树和右子树分别又是一棵二叉树。

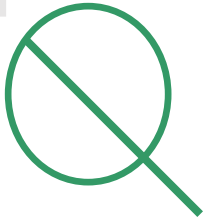
特点：每个结点至多只有两棵子树，且二叉树的子树有左右之分，其次序不能任意颠倒。




根结点
右子树
A
B
C
D
E
F
G
H
K
左子树

二、二叉树的五种基本形态：

空树

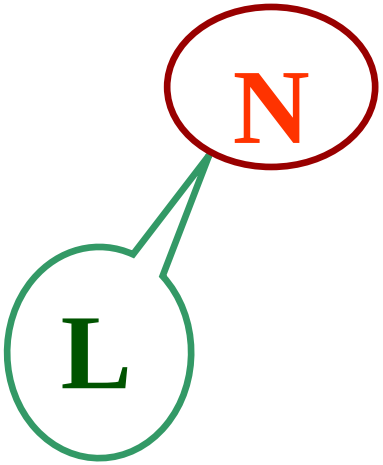


只含根结点

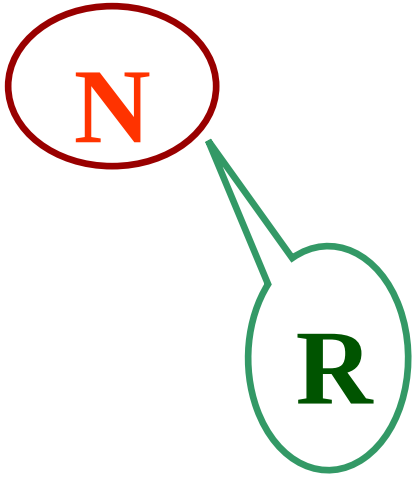


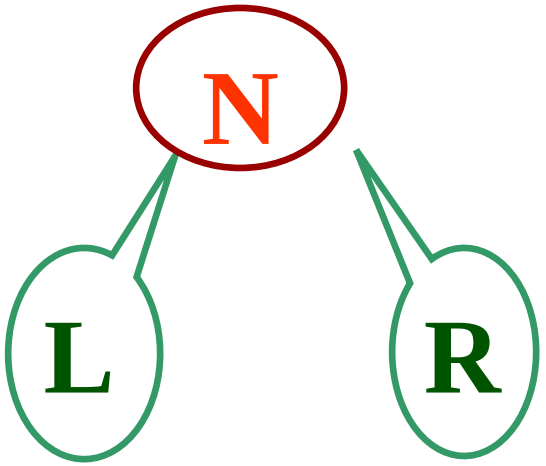
左右子树均不为空树

右子树为空树



左子树为空树





三、二叉树的主要基本操作：



查 找 类



插 入 类



删 除 类

查找类：

Root(T) //求根函数；
Value(T, e)；//求结点函数；
Parent(T, e)；//求双亲函数；
LeftChild(T, e)或
    RightChild(T, e)；//求孩子结点函数；
LeftSibling(T, e)或
    RightSibling(T, e)；//求兄弟函数；
BiTreeEmpty(T)；//树判空函数；
BiTreeDepth(T)；//求树深度函数；
PreOrderTraverse(T, Visit())；//先序边历操作
InOrderTraverse(T, Visit())；//中序边历操作
PostOrderTraverse(T, Visit())；//后序边历操作

插入类：

```
InitBiTree(&T); //构造空的二叉树;

Assign(T, &e, value); //结点赋值函数

CreateBiTree(&T, definition); //建树函数;

InsertChild(T, p, LR, c); //插入子树操作;
```

删除类：

```
ClearBiTree(&T); // 将二叉树置为空树

DestroyBiTree(&T); // 销毁二叉树

DeleteChild(T, p, LR); // 删除子树操作
```

四、二叉树的重要特性：

性质1：

在二叉树的第 i 层上至多有 2^{i-1} 个结点。 ($i \geq 1$)

用归纳法证明：

归纳基：　$i = 1$ 层时，只有一个根结点：

$$2^{i-1} = 2^0 = 1;$$

归纳假设：

归纳证明：　假设对所有的 j，$1 \leq j < i$，命题成立；

二叉树上每个结点至多有两棵子树，

则第 i 层的结点数 $= 2^{i-2} \times 2 = 2^{i-1}$ 。

性质 2：

深度为 k 的二叉树上至多含 2^k-1 个结点（k≥1）

证明：

基于上一条性质，深度为 k 的二叉树上的结点数至多为

$2^0+2^1+\cdots\cdots+2^{k-1}=2^k-1$ 。

性质 3 ：

对任何一棵二叉树，若它含有 n_0 个叶子结点、n_2 个度为 2 的结点，则必存在关系式：$n_0=n_2+1$。

证明：

设 二叉树上结点总数 $n = n_0 + n_1 + n_2$

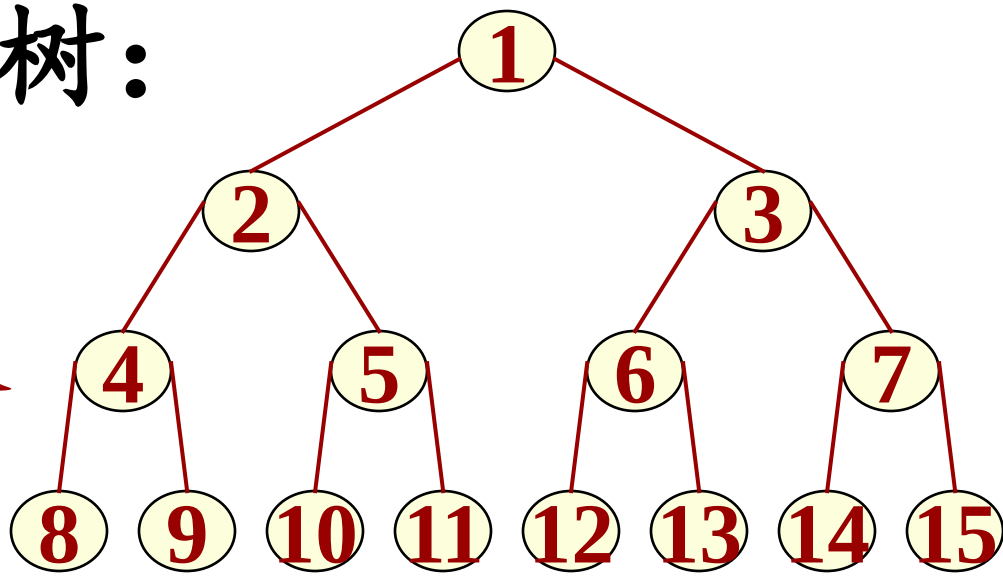
又 二叉树上分支总数 $b = n_1+2n_2$

而 $b = n-1 = n_0 + n_1 + n_2 - 1$

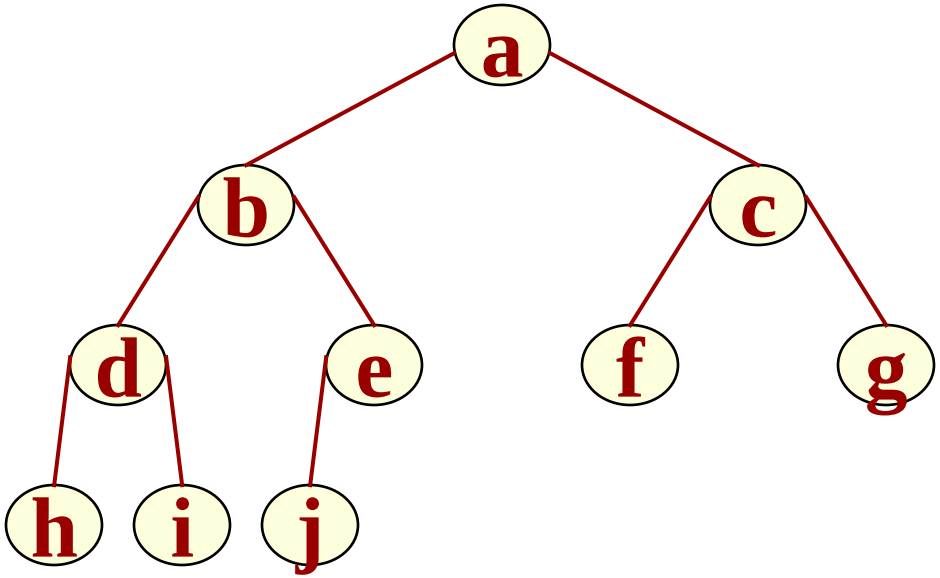
由此， $n_0 = n_2 + 1$ 。

两类特殊的二叉树：

满二叉树：指的是深度为k且含有2^k-1个结点的二叉树。



完全二叉树：树中所含的 n 个结点和满二叉树中编号为 1 至 n 的结点一一对应。



性质 4 ：

具有n个结点的完全二叉树的深度为$\lfloor \log_2 n \rfloor + 1$。

证明：

设完全二叉树的深度为 k

则根据第二条性质得 $2^{k-1} \leq n < 2^k$

即 $k-1 \leq \log_2 n < k$

因为 k 只能是整数，因此，$k = \lfloor \log_2 n \rfloor + 1$。

性质 5 ：

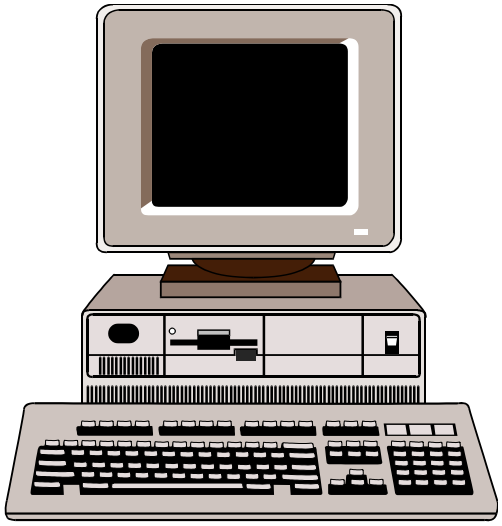
若对含 n 个结点的完全二叉树从上到下且从左至右进行 1 至 n 的编号，则对完全二叉树中任意一个编号为 i 的结点：

(1) 若 $i=1$，则该结点是二叉树的根，无双亲，否则，编号为 $\lfloor i/2 \rfloor$ 的结点为其**双亲**结点；

(2) 若 $2i>n$，则该结点无左孩子，否则，编号为 $2i$ 的结点为其**左孩子**结点；

(3) 若 $2i+1>n$，则该结点无右孩子结点，否则，编号为 $2i+1$ 的结点为其**右孩子**结点。

6.3 二叉树的存储结构

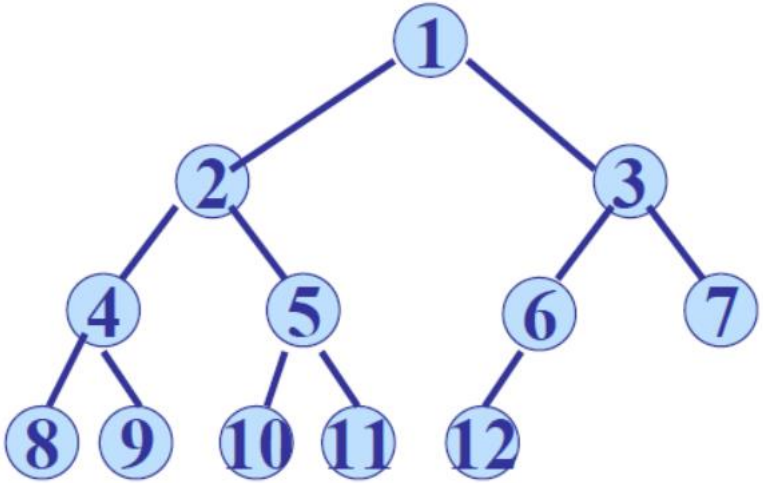


一、 二叉树的顺序存储表示

二、二叉树的链式存储表示

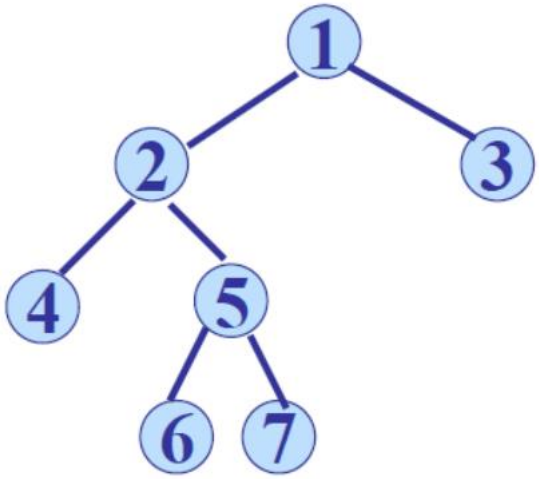
一、 二叉树的顺序存储表示

用一组地址连续的存储单元依次自上而下、自左至右存储完全二叉树上的结点元素。对于一般二叉树，则应将其每个结点与完全二叉树上的结点相对照，存储在一维数组的相应分量中。



1	2	3	4	5	6	7	8	9	10	11	12

完全二叉树



1	2	3	4	5	0	0	0	0	6	7

一般二叉树

```
#define  MAX_TREE_SIZE  100
                // 二叉树的最大结点数
typedef TElemType  SqBiTree[MAX_
                TREE_SIZE];
                // 0号单元存储根结点
SqBiTree  bt;
```

二、二叉树的链式存储表示

1. 二叉链表　　3. 双亲链表

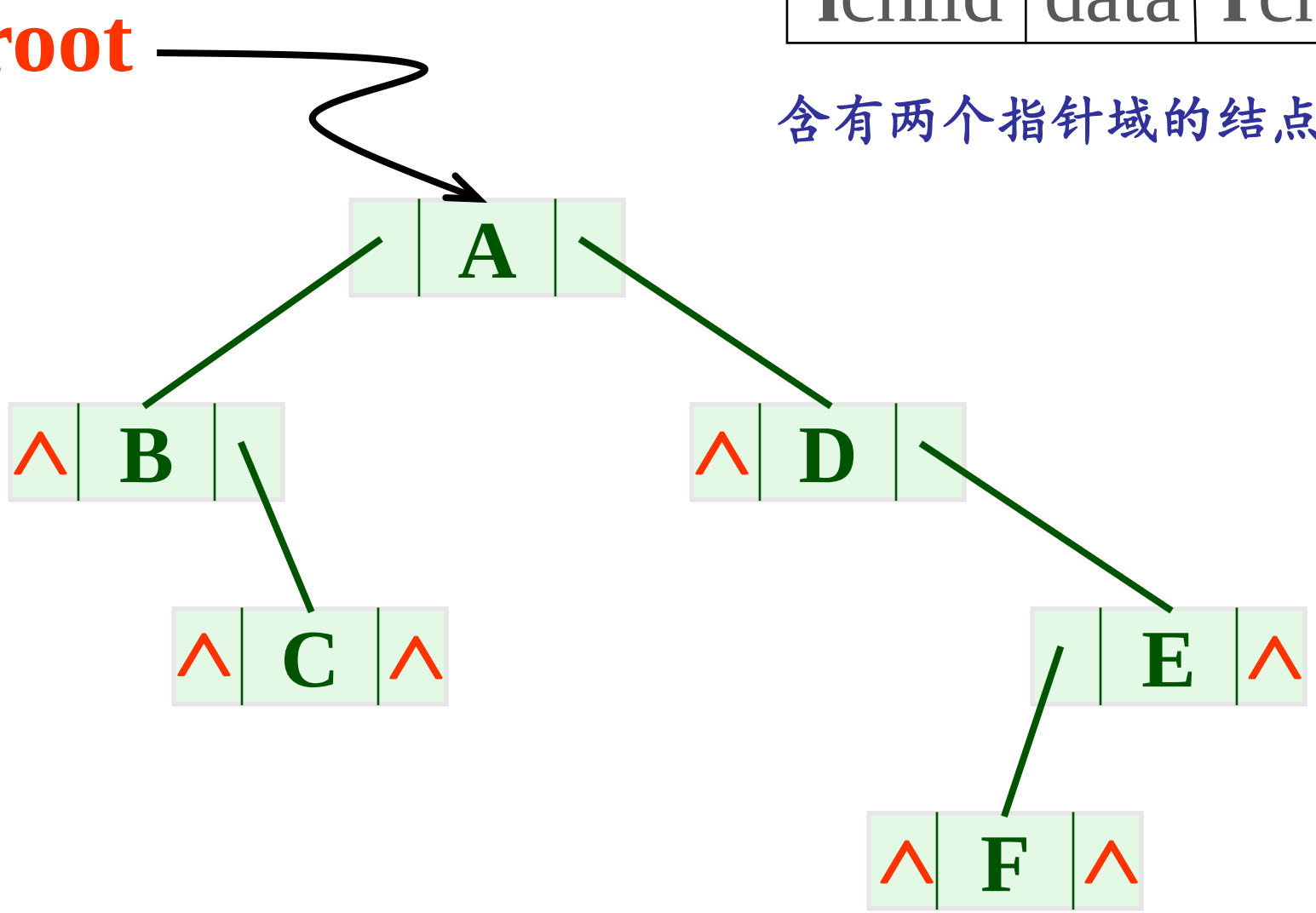
2. 三叉链表　　4. 线索链表

1. 二叉链表

结点结构:

lchild	data	rchild

含有两个指针域的结点结构



C 语言的类型描述如下：

```
typedef struct BiTNode { // 结点结构
    TElemType      data;
    struct BiTNode  *lchild, *rchild;
                                  // 左右孩子指针
} BiTNode, *BiTree;
```

结点结构：

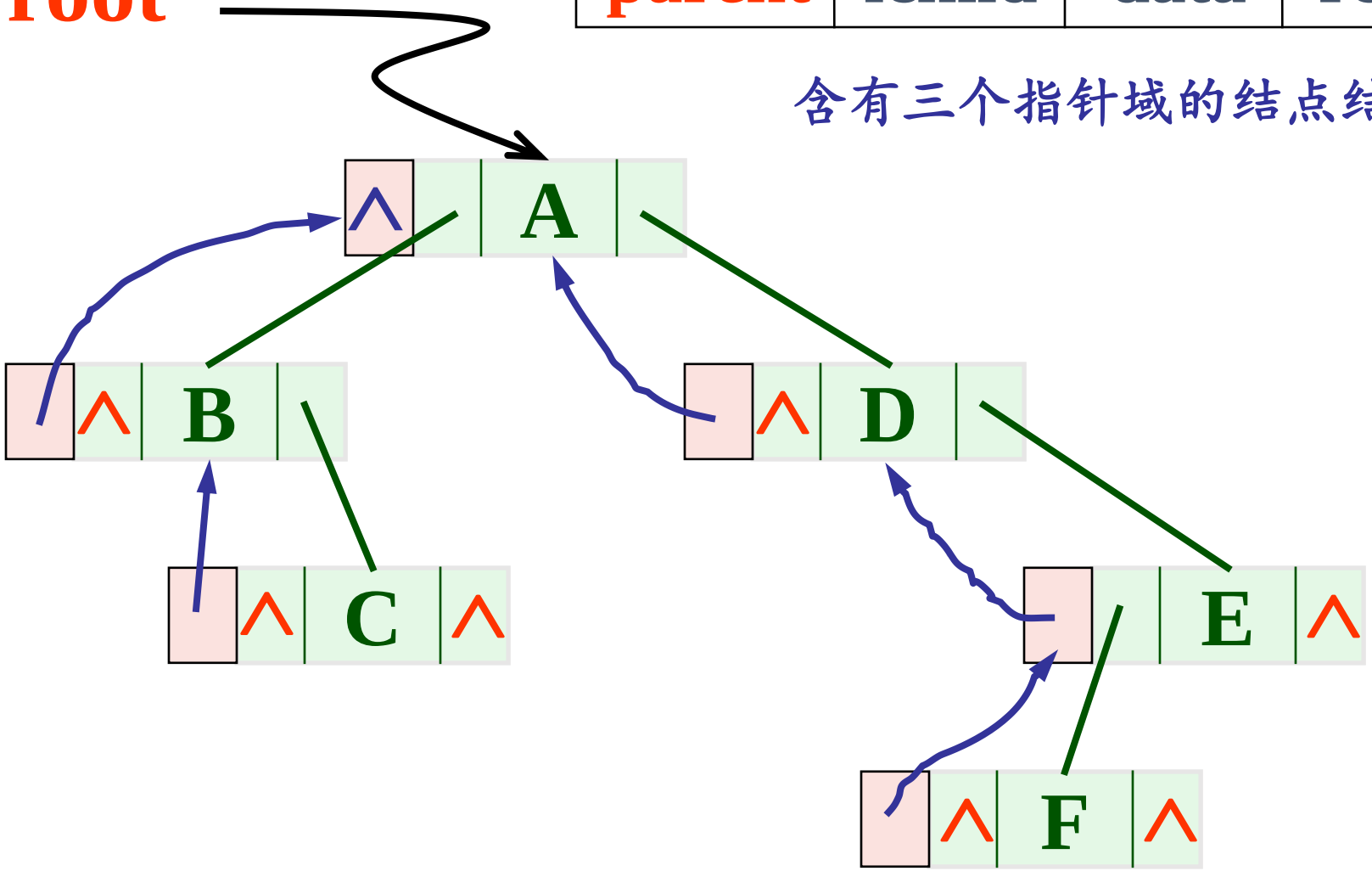


2. 三叉链表

结点结构:

parent	lchild	data	rchild

含有三个指针域的结点结构

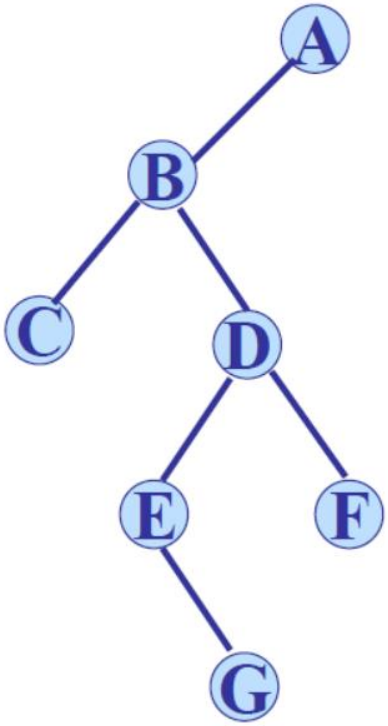


C 语言的类型描述如下：

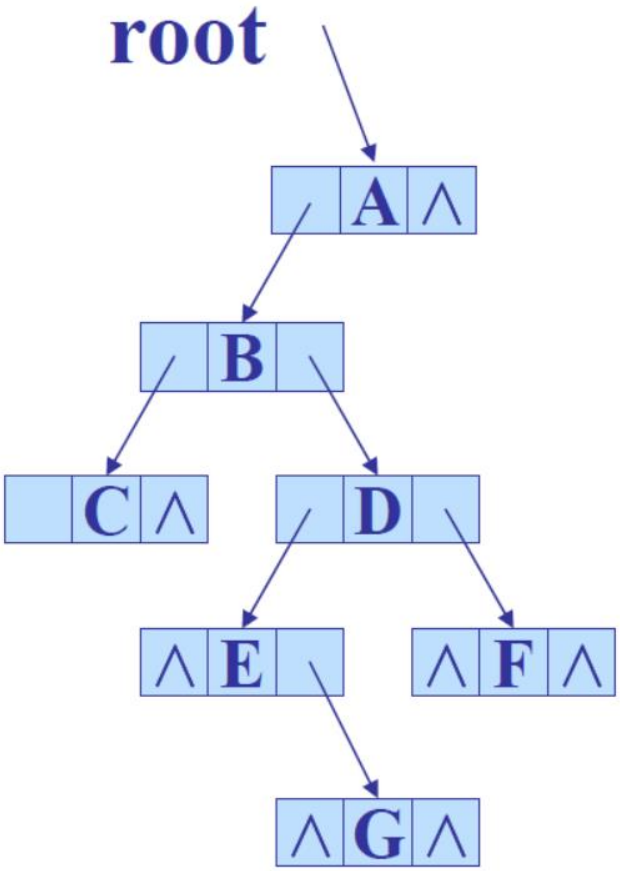
```
typedef struct TriTNode { // 结点结构
    TElemType       data;
    struct TriTNode  *lchild, *rchild;
                                  // 左右孩子指针
    struct TriTNode  *parent;  //双亲指针
} TriTNode, *TriTree;
```

结点结构：

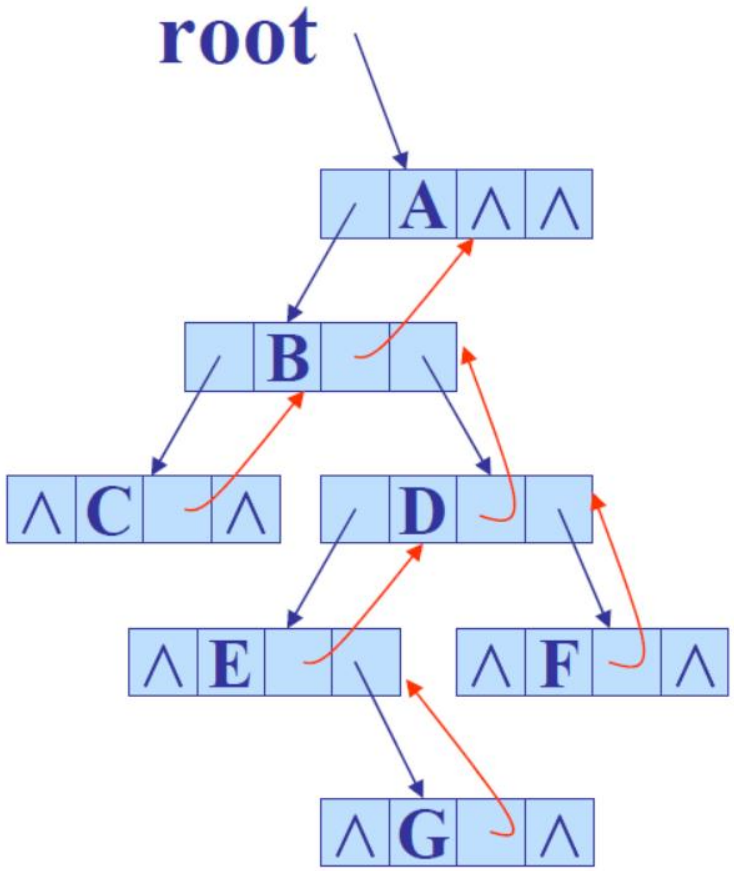
parent	lchild	data	rchild



二叉树



二叉链表

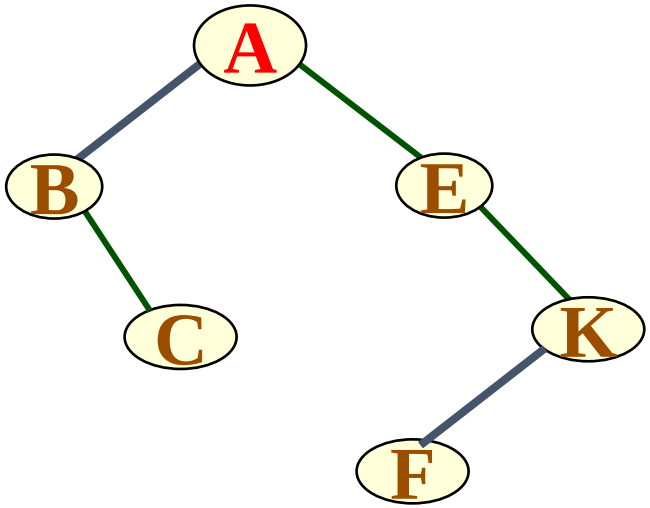


三叉链表

3. 双亲链表

结点结构:

data	parent	LRTag



0	B	2	L
1	C	0	R
2	A	-1	
3	D	2	R
4	E	3	R
5	F	4	L
6			

```
typedef struct BPTNode { // 结点结构
    TElemType  data;
    int  parent;     // 指向双亲的指针
    char  LRTag;    // 左、右孩子标志域
} BPTNode
typedef struct BPTree{ // 树结构
    BPTNode nodes[MAX_TREE_SIZE];
    int  num_node;     // 结点数目
    int  root;                 // 根结点的位置
} BPTree
```

6.4 二叉树的遍历

一、问题的提出

二、先左后右的遍历算法

三、算法的递归描述

四、中序遍历算法的非递归描述

一、问题的提出

顺着某一条搜索路径**巡访**二叉树中的结点，使得每个结点**均被访问一次**，而且**仅被访问一次**。

“**访问**”的含义可以很广，如：输出结点的信息等。

“**遍历**”是任何类型均有的操作，对线性结构而言，只有一条搜索路径(因为每个结点均只有一个后继)，故不需要另加讨论。而二叉树是非线性结构，**每个结点有两个后继**，则**存在如何遍历**即按什么样的**搜索路径**遍历的问题。

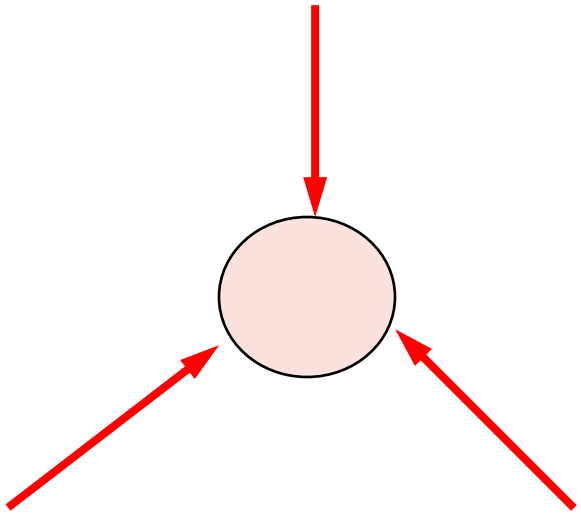
对“二叉树”而言，可以有三条搜索路径：

1. **先上后下**的按层次遍历；

2. **先左**（子树）**后右**（子树）的遍历；

3. **先右**（子树）**后左**（子树）的遍历。

二、先左后右的遍历算法



先（根）序的遍历算法

中（根）序的遍历算法

后（根）序的遍历算法

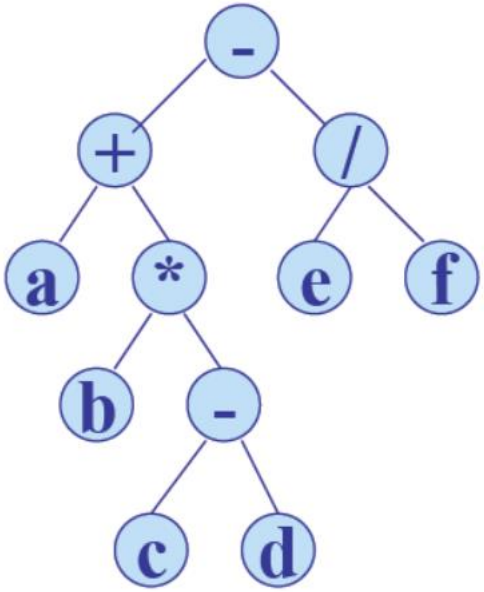
先（根）序的遍历算法：

若二叉树为空树，则空操作；否则，

（1）访问根结点；

（2）先序遍历左子树；

（3）先序遍历右子树。



先序：-+a*b-cd/ef

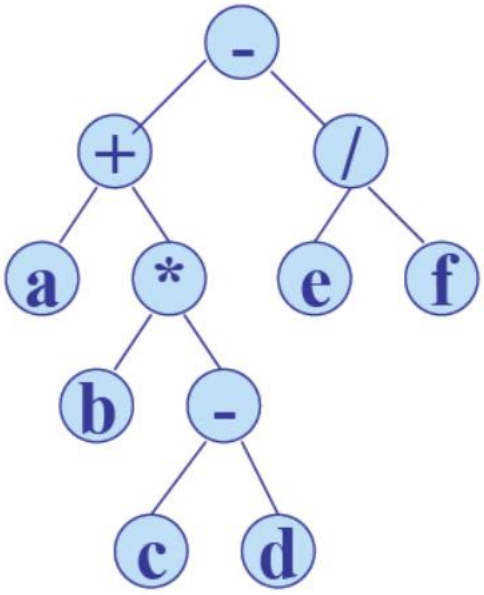
中（根）序的遍历算法：

若二叉树为空树，则空操作；否则，

（1）中序遍历左子树；

（2）访问根结点；

（3）中序遍历右子树。



中序：a+b*c-d-e/f

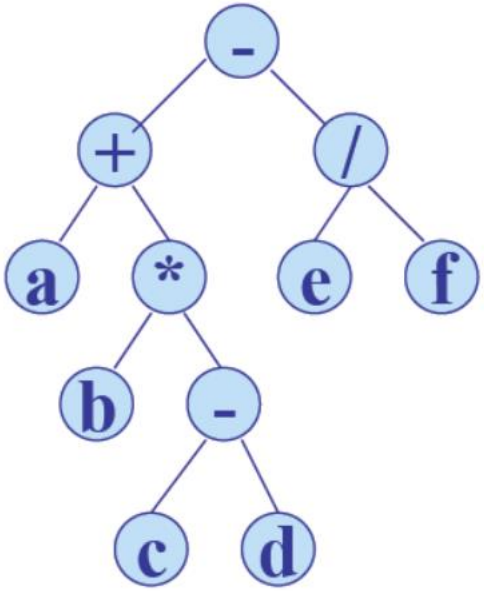
后（根）序的遍历算法：

若二叉树为空树，则空操作；否则，

（1）后序遍历左子树；

（2）后序遍历右子树；

（3）访问根结点。



后序：abcd-*+ef/-

三、算法的递归描述

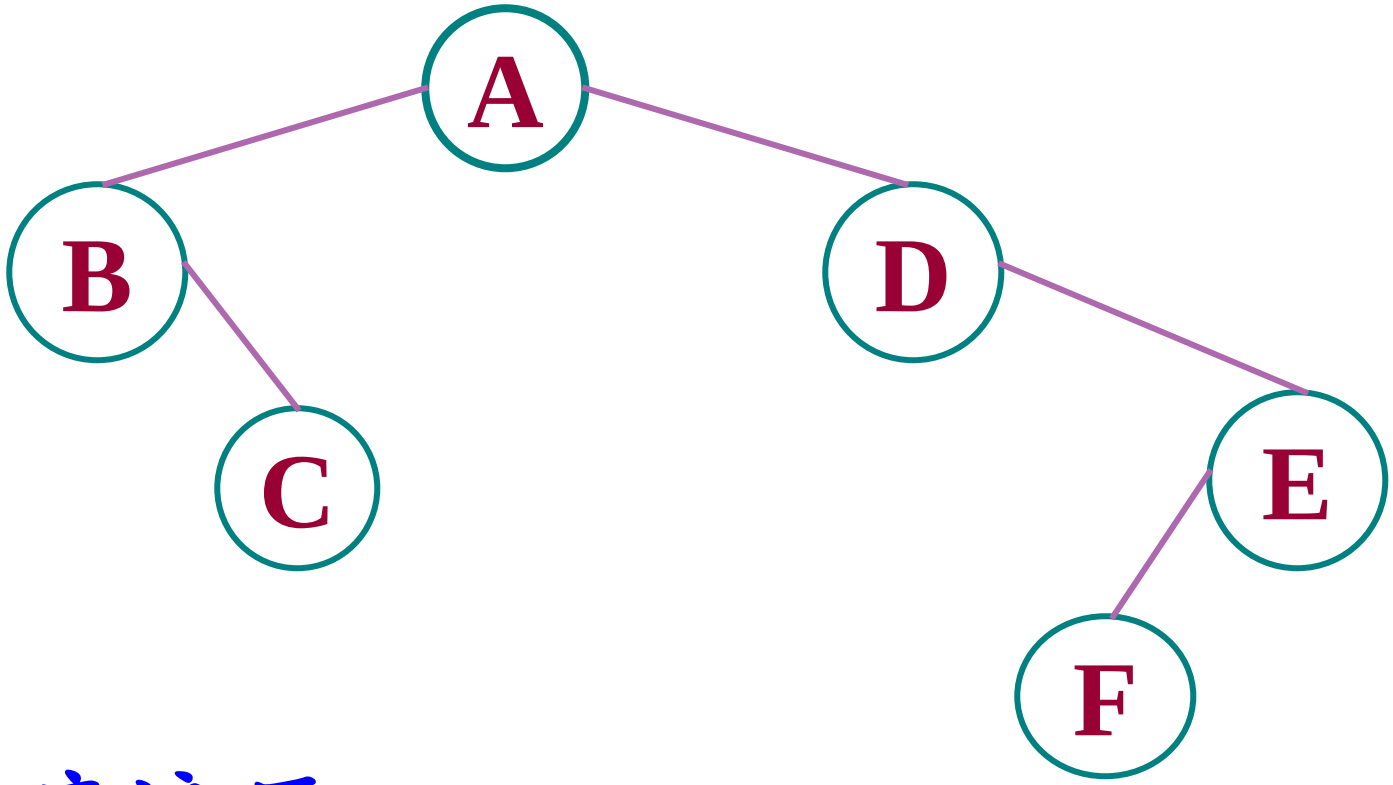
```
void Preorder (BiTree T,
               void( *visit)(TElemType& e))
{                                  // 先序遍历二叉树
  if (T) {
    visit(T->data);                // 访问结点
    Preorder(T->lchild, visit); // 遍历左子树
    Preorder(T->rchild, visit);// 遍历右子树
  }
}
```

```
void Inorder (BiTree T,
              void( *visit)(TElemType& e))
{                                  // 先序遍历二叉树
  if (T) {
    Inorder(T->lchild, visit); // 遍历左子树
    visit(T->data);            // 访问结点
    Inorder(T->rchild, visit);// 遍历右子树
  }
}
```

```
void Postorder (BiTree T,
                void( *visit)(TElemType& e))
{                                  // 先序遍历二叉树
  if (T) {
    Postorder(T->lchild, visit); // 遍历左子树
    Postorder(T->rchild, visit);// 遍历右子树
    visit(T->data);              // 访问结点  }
}
```

递归算法简明精练，但效率较低。

四、算法的非递归描述



中序遍历：BCADFE

中序遍历算法的非递归描述

```
BiTNode *GoFarLeft(BiTree T, Stack *S){
   if (!T )  return NULL;
   while (T->lchild ){
      Push(S, T);
      T = T->lchild;
   }
   return T;
}
```

```
void Inorder_I(BiTree T, void (*visit)
                              (TelemType& e)){
   Stack *S;
   t = GoFarLeft(T, S);  // 找到最左下的结点
   while(t){
     visit(t->data);
     if (t->rchild)
          t = GoFarLeft(t->rchild, S);
     else if ( !StackEmpty(S))    // 栈不空时退栈
          t = Pop(S);
     else    t = NULL; // 栈空表明遍历结束
    } // while
}// Inorder_I
```

第七次作业：

实现求二叉树深度的算法：

int TreeDepth(BiTree T)

```
typedef struct BiTNode { // 结点结构
    TElemType      data;
    struct BiTNode  *lchild, *rchild;   // 左右孩子指针
} BiTNode, *BiTree;
```

6.5 线索二叉树

一．何谓线索二叉树?

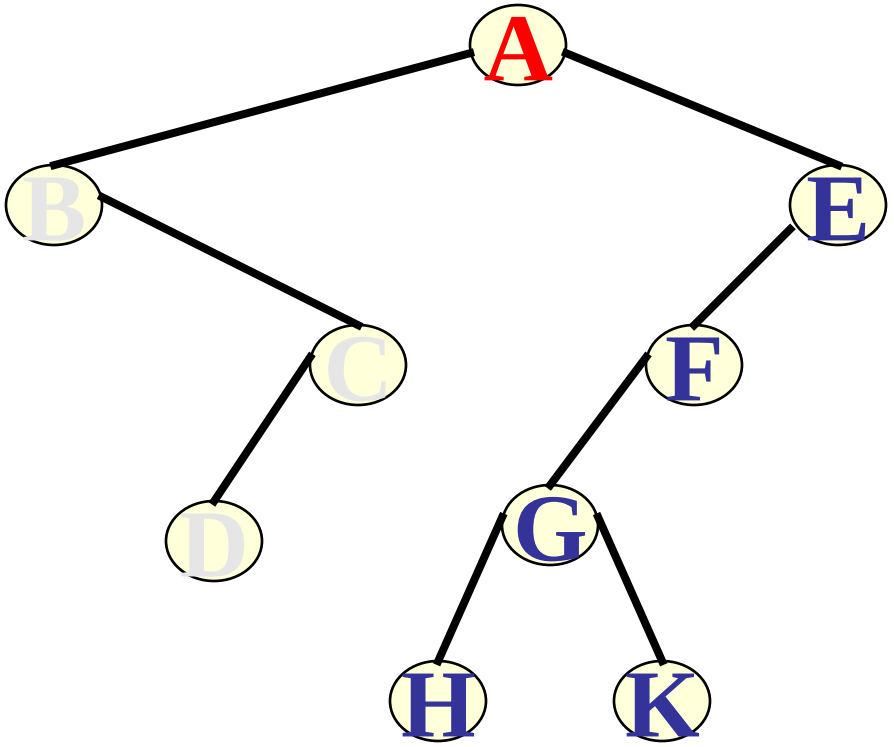
二．线索链表的遍历算法

三．如何建立线索链表?

一、何谓线索二叉树?

遍历二叉树的结果是,

求得结点的一个线性序列。



例如:

先序序列:

A B C D E F G H K

中序序列:

B D C A H G K F E

后序序列:

D C B H K G F E A

当以二叉链表作为存储结构时，只能找到结点的左右孩子信息，而不能直接得到结点在任一序列的前驱和后继信息，这种信息只有在遍历的动态过程中才能得到。

如何保存这种在遍历过程中得到的信息呢？

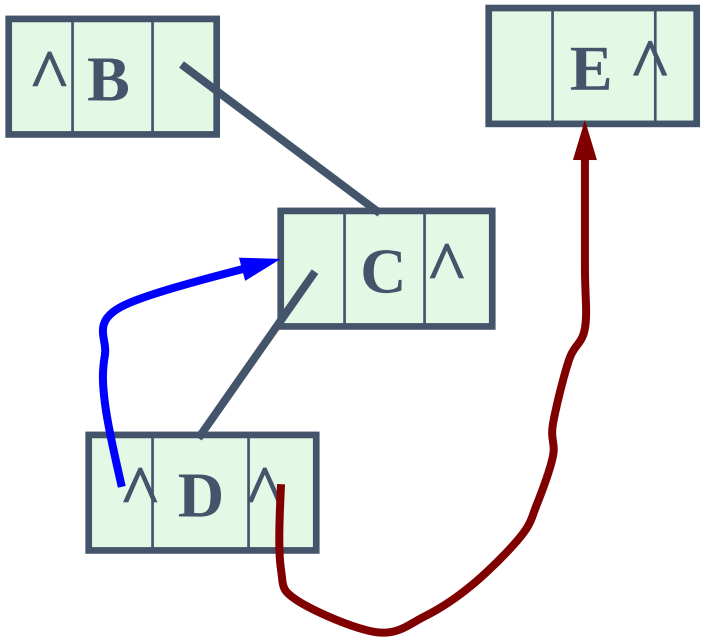
方法一：在每个结点上增加两个指针域，分别指示结点在任一次序遍历时得到的前驱和后继信息。

显然，这样做使得结构的存储密度大大降低。

方法二：在有n个结点的二叉链表中必定存在n+1个空链域，利用这些空链域来存放结点的前驱和后继的信息。

- 指向该线性序列中的“前驱”和“后继”的**指针**，称作“**线索**”

A B C D E F G H K



包含“线索”的存储结构，称作“**线索链表**”

与其相应的二叉树，称作“**线索二叉树**”

对线索链表中结点的约定：

在二叉链表的结点中**增加两个标志域**，并作如下规定：

- **若该结点的左子树不空，**

  则Lchild域的指针指向其左子树，

  且左标志域的值为“指针 Link”；

  否则，Lchild域的指针指向其“前驱”，

  且左标志的值为“线索 Thread”。

- 若该结点的右子树不空，

  则rchild域的指针指向其右子树，

  且右标志域的值为“指针 Link”；

  否则，rchild域的指针指向其“后继”，

  且右标志的值为“线索 Thread”。

如此二叉树的存储结构称作“线索链表”

对二叉树以某种次序遍历使其变为线索二叉树的过程，叫做线索化。

线索链表的类型描述：

```
typedef enum { Link, Thread } PointerThr;
        // Link==0:指针，Thread==1:线索

typedef struct BiThrNod {
   TElemType       data;
   struct BiThrNode  *lchild, *rchild;  // 左右指针
   PointerThr        LTag, RTag;    // 左右标志
} BiThrNode, *BiThrTree;
```

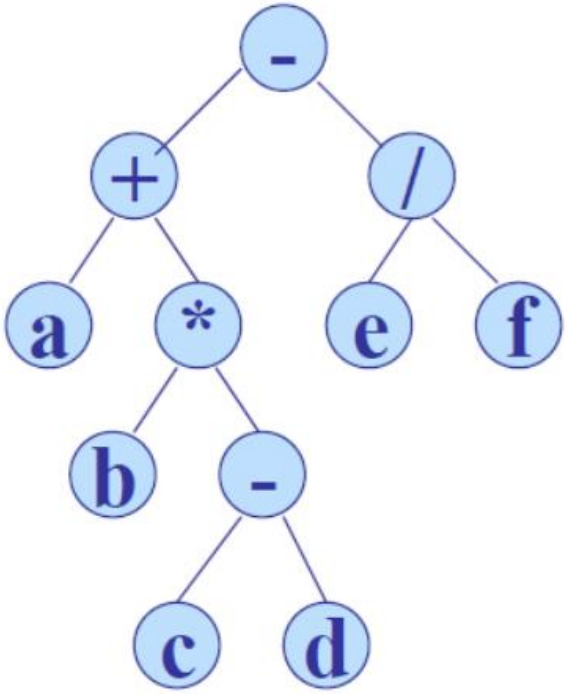
lchild	ltag	data	rtag	rchild

结点结构

其中：

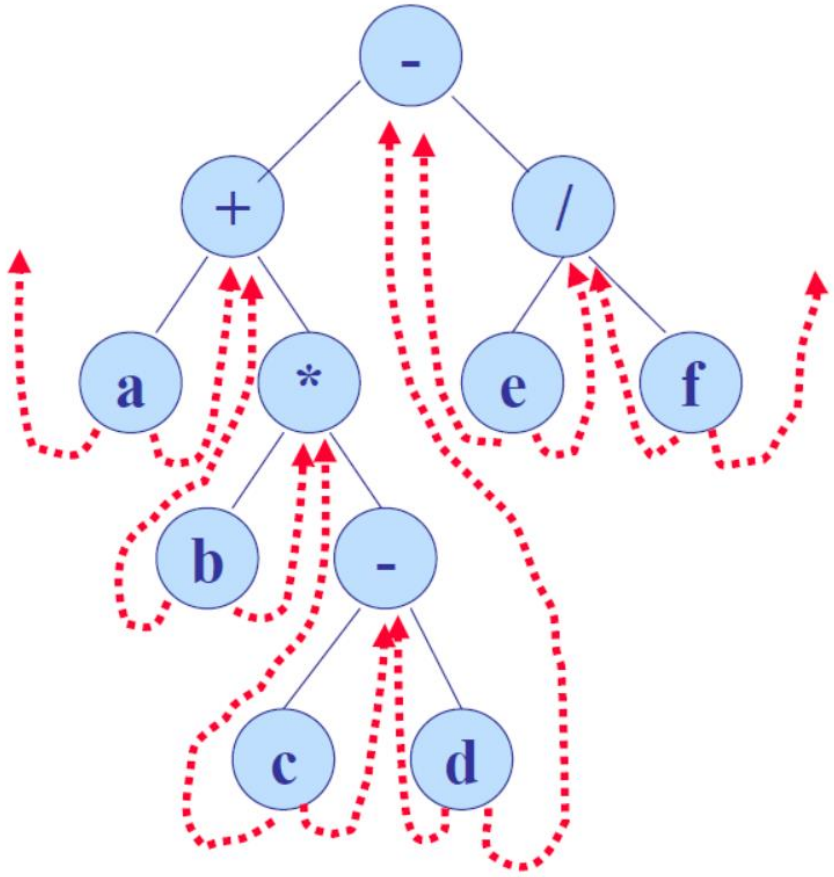
ltag= $\begin{cases} 0 & \text{lchild域指示结点的左孩子} \\ 1 & \text{lchild域指示结点的前趋} \end{cases}$

rtag= $\begin{cases} 0 & \text{rchild域指示结点的右孩子} \\ 1 & \text{rchild域指示结点的后继} \end{cases}$



中序序列:

a+b*c-d-e/f



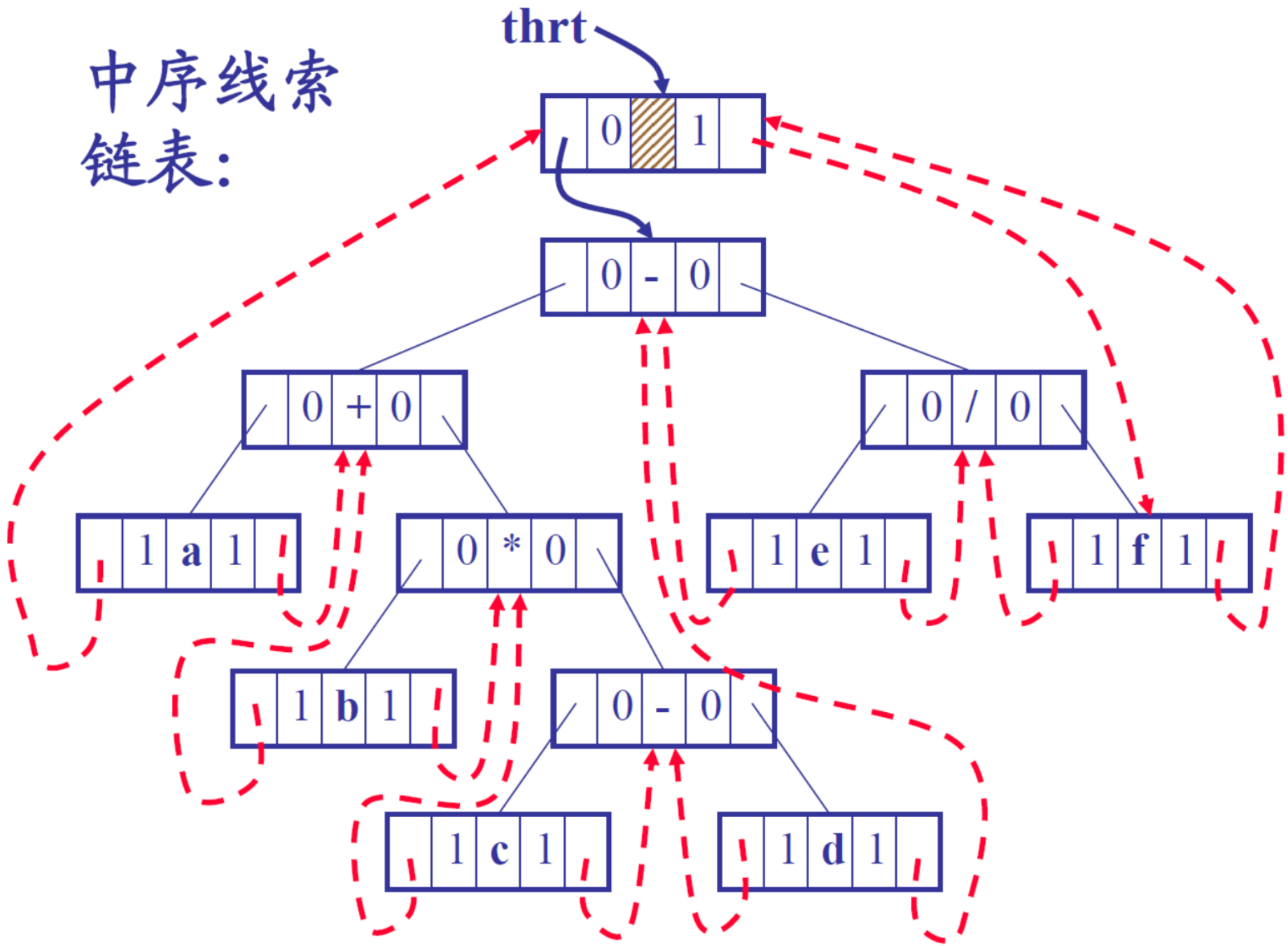
中序线索二叉树

为方便，仿照线性表的存储结构，在二叉树的线索链表上也添加一个头结点，并令其lchild域的指针指向二叉树的根结点，其rchild域的指针指向中序遍历时访问的最后一个结点；

反之，令二叉树中序序列中的第一个结点的lchild域指针和最后一个结点rchild域的指针均指向头结点。

这好比为二叉树建立了一个双向线索链表，既可从第一个结点起顺后继进行遍历，也可从最后一个结点起顺前驱进行遍历。

中序线索链表:



二、线索链表的遍历算法

由于在线索链表中添加了遍历中得到的“前驱”和“后继”的信息，从而简化了遍历的算法。

```
for ( p = firstNode(T); p; p = Succ(p) )
    Visit (p);
```

例如：对中序线索化链表的遍历算法

※ 中序遍历的第一个结点 ？

左子树上处于“最左下”（没有左子树）的结点。

※ 在中序线索化链表中结点的后继 ？

若无右子树，则为后继线索所指结点；

否则为对其右子树进行中序遍历时访问的第一个结点。

```
void InOrderTraverse_Thr(BiThrTree T,
                         void (*Visit)(TElemType e)) {
  p = T->lchild;        // p指向根结点
  while (p != T) {      //空树或遍历结束时，p==T
    while (p->LTag==Link)  p = p->lchild;  //第一个结点
    Visit(p->data);
    while (p->RTag==Thread && p->rchild!=T)
    {  p = p->rchild;  Visit(p->data);  }  //访问后继结点
    p = p->rchild;          // p进至其右子树根
  }
} // InOrderTraverse_Thr
```

三、如何建立线索链表？

线索化的实质是将二叉链表中的空指针改为指向前驱或后继的线索，而前驱或后继的信息只有在遍历时才能得到。因此，线索化的过程即为在遍历过程中修改空指针的过程。

为了记下遍历过程中访问结点的先后关系，附设一个指针pre，并始终保持指针pre指向当前访问的、指针p所指结点的前驱。

```
void InThreading(BiThrTree p) {
  if (p) {    // 对以p为根的非空二叉树进行线索化
     InThreading(p->lchild);      // 左子树线索化
     if (!p->lchild)      // 建前驱线索
        { p->LTag = Thread;    p->lchild = pre; }
     if (!pre->rchild)   // 建后继线索
        { pre->RTag = Thread;   pre->rchild = p; }
     pre = p;                  // 保持 pre 指向 p 的前驱
     InThreading(p->rchild);      // 右子树线索化
  } // if
} // InThreading
```

```
Status InOrderThreading(BiThrTree &Thrt,
                        BiThrTree T) { //构建中序线索链表
  if (!(Thrt = (BiThrTree)malloc(sizeof( BiThrNode))))
    exit (OVERFLOW);
  Thrt->LTag = Link;  Thrt->RTag =Thread;
  Thrt->rchild = Thrt;      // 添加头结点
          … …
  return OK;
} // InOrderThreading
```

```
if (!T)  Thrt->lchild = Thrt;
else {
   Thrt->lchild = T;     pre = Thrt;
   InThreading(T);
   pre->rchild = Thrt;   // 处理最后一个结点
   pre->RTag = Thread;
   Thrt->rchild = pre;
}
```

第八次作业：

若某二叉树的先序遍历序列**eadcbjfglh**与中序遍历序列**acbdjeflgh**，写出后序遍历序列，并画出它的带头节点的后序线索二叉树。

6.6 树和森林

一、树的存储结构

二、森林与二叉树的转换

一、树的三种存储结构

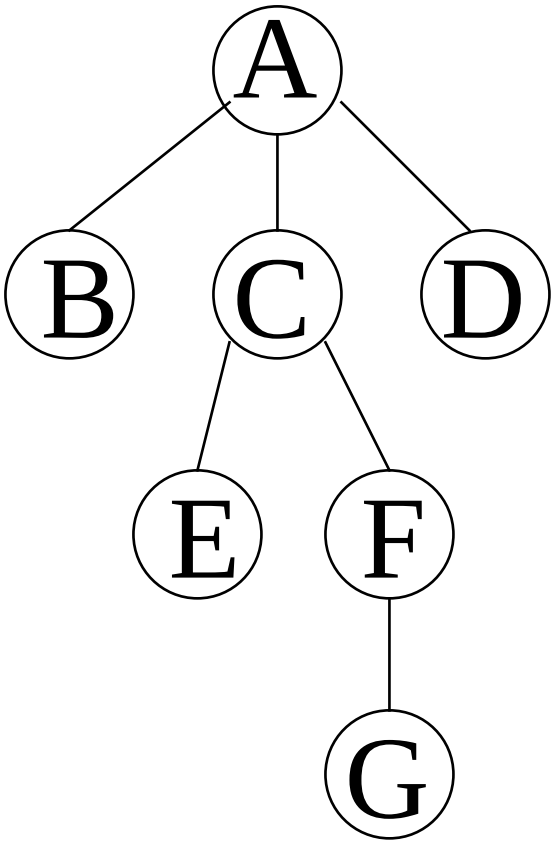
1、双亲表示法

2、孩子链表表示法

3、树的二叉链表(孩子-兄弟)
存储表示法

1、双亲表示法:

以一组连续空间存储树的结点，同时在每个结点中附设一个指示器指示其双亲结点的位置。



	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	2
5	F	2
6	G	5

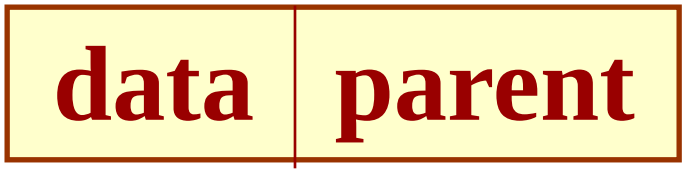
r=0

n=7

C语言的类型描述：

```
#define MAX_TREE_SIZE  100
```

结点结构：




```
typedef struct PTNode {
    Elem  data;
    int    parent;   // 双亲位置域
} PTNode;
```

树结构:

```
typedef struct {
    PTNode  nodes[MAX_TREE_SIZE];
    int     r, n;
            // 根结点的位置和结点个数
  } PTree;
```

这种存储结构利用了每个结点（除根外）只有唯一双亲的性质。**PARENT(T, x)**操作可在常量时间内实现。反复调用**PARENT**操作，直到遇见无双亲的结点时，便找到了树的根，这就是**ROOT(x)**操作的执行过程。

但是，在这种表示法中，求结点的孩子时需要遍历整个结构。

2、孩子链表表示法：

由于树中每个结点可能有多棵子树，则在采用链式存储结构时可用多重链表，即每个结点有多个指针域，其中，每个指针指向一棵子树的根结点。

此时，链表中的结点可有如下两种结点格式：

data	child1	child2	…	childd

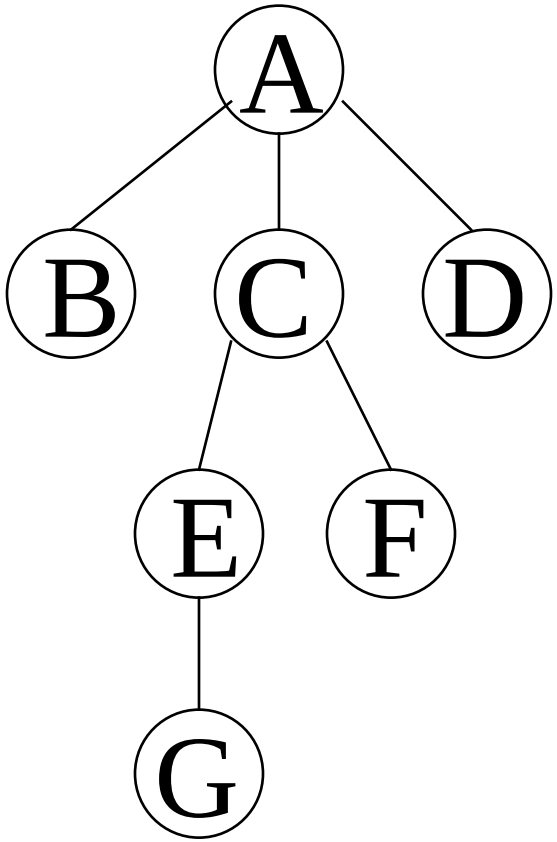
data	degree	child1	child2	…	Childd

若采用第一种结点格式，则多重链表中的结点是同构的，其中**d**为树的度。由于树中很多结点的度小于**d**，所以链表中有很多空链域，空间较浪费。不难推出，在一棵有**n**个结点度为**k**的树中必有**n(k−1)+1**个空链域。

若采用第二种结点格式，则多重链表中的结点是不同构的，其中**d**为结点的度，**degree**域的值同**d**。此时，虽能节省存储空间，但操作不方便。

另一种办法是把每个结点的孩子结点排列起来，看成是一个线性表，且以单链表作存储结构，则n个结点有n个孩子链表（叶子的孩子链表为空表）。

而n个头指针又组成一个线性表，为了便于查找，可采用顺序存储结构。



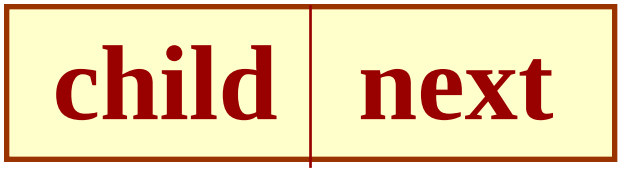
	data	firstchild
0	**A**	→ 1 → 2 → 3 ∧
1	**B**	∧
2	**C**	→ 4 → 5 ∧
3	**D**	∧
4	**E**	→ 6 ∧
5	**F**	∧
6	**G**	∧

r=0

n=7

C语言的类型描述：

孩子结点结构：




```
typedef struct CTNode {
    int         child;
    struct      CTNode *next;
} *ChildPtr;
```

双亲结点结构

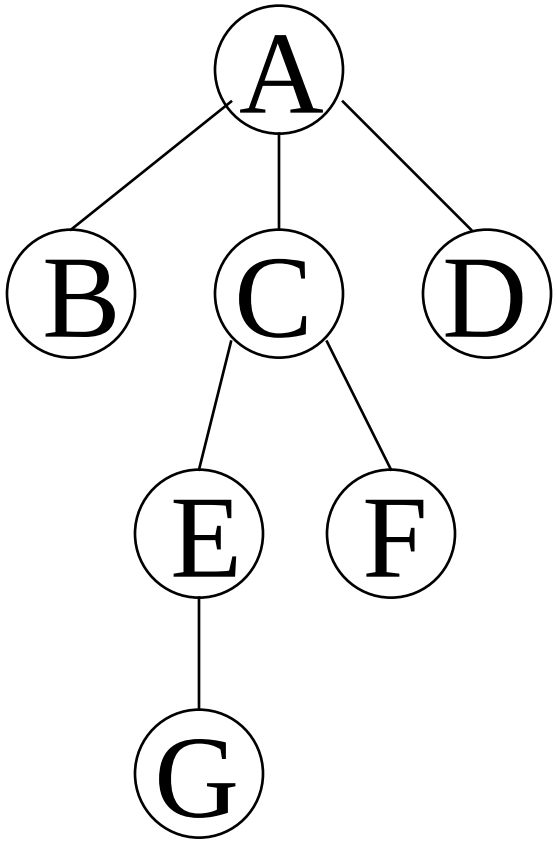



```
typedef struct {
  Elem    data;
  ChildPtr  firstchild;
                  // 孩子链的头指针
} CTBox;
```

树结构:

```
typedef struct {
    CTBox  nodes[MAX_TREE_SIZE];
    int    n, r;  // 结点数和根结点的位置
  } CTree;
```

与双亲表示法相反，孩子表示法便于那些涉及孩子的操作的实现，却不适用于**PARTEN(x)**操作。可把双亲表示法和孩子链表合在一起，组成带双亲的孩子链表。



	data	firstchild	
0	**A**	-1	→ 1 → 2 → 3 ∧
1	**B**	0	∧
2	**C**	0	→ 4 → 5 ∧
3	**D**	0	∧
4	**E**	2	→ 6 ∧
5	**F**	2	∧
6	**G**	4	∧

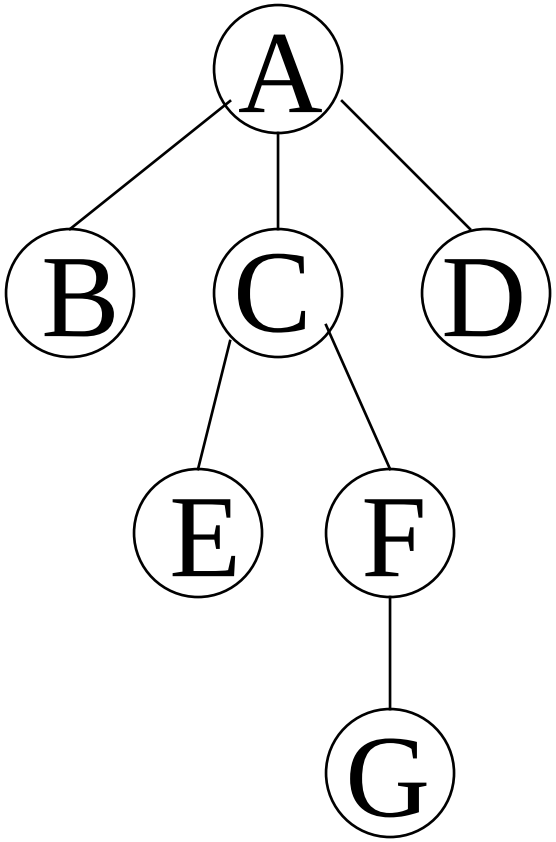
r=0

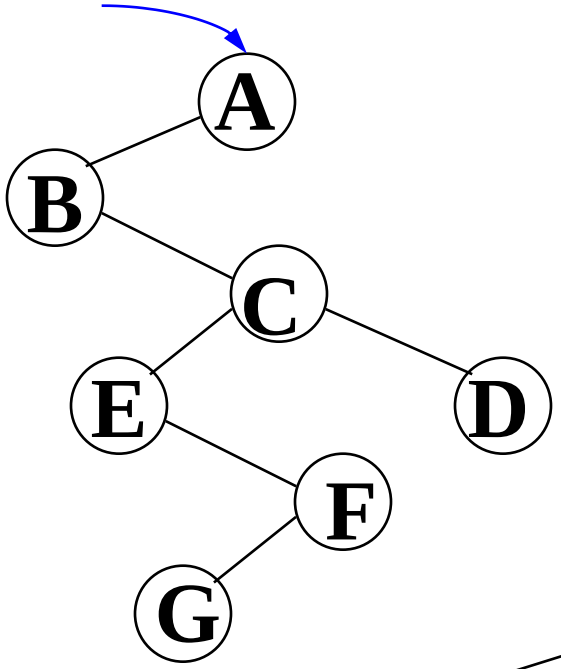
n=7

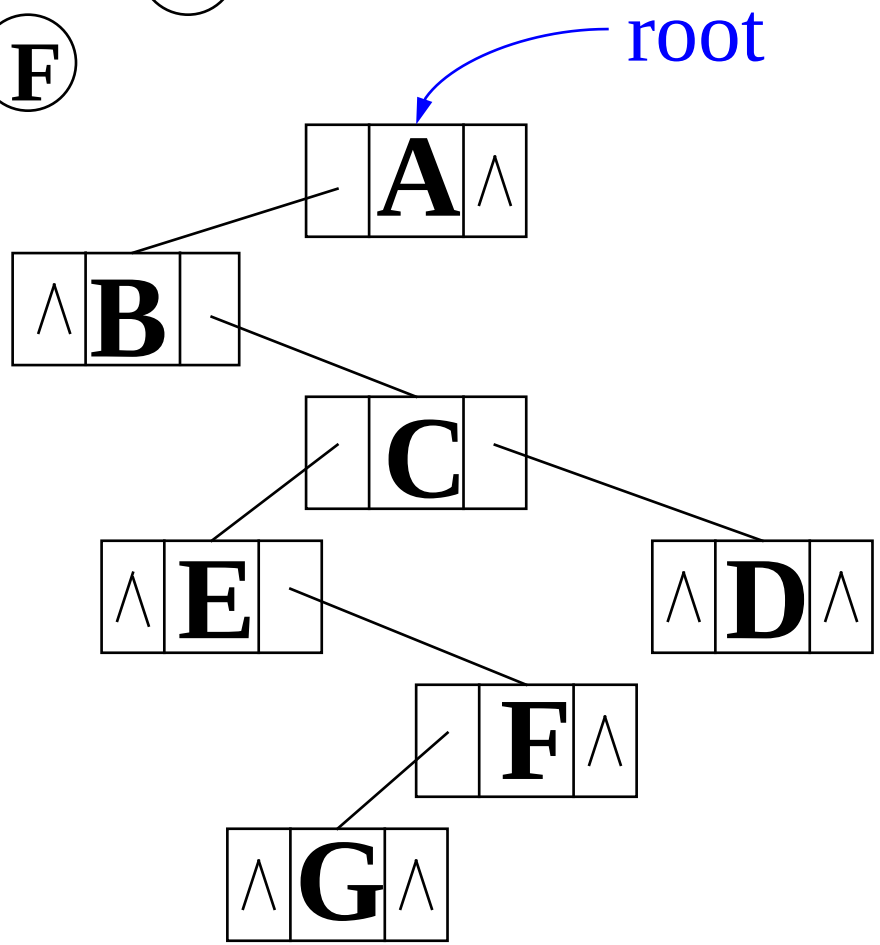
3、树的二叉链表 (孩子-兄弟）存储表示法

又称二叉树表示法，或二叉链表表示法。即以二叉链表作树的存储结构。

链表中结点的两个链域分别指向该结点的第一个孩子结点和下一个兄弟结点。


A
B
C
D
E
F
G


A
B
C
E
D
F
G


root
A
B
C
E
D
F
G

C语言的类型描述：

结点结构：




```
typedef struct CSNode{
    Elem        data;
    struct CSNode
            *firstchild, *nextsibling;
} CSNode, *CSTree;
```

利用这种存储结构便于实现各种树的操作。首先易于实现找结点孩子等的操作。

例如，若要访问结点x的第i个孩子，则只要先从firstchild域找到第1个孩子，然后沿着孩子结点的nextsibling域连续走i-1步，便可找到x的第i个孩子。

由此，树的各种操作均可对应二叉树的操作来完成。

应当注意的是，和树对应的二叉树，其左、右子树的概念

已改变为：左是孩子，右是兄弟。

二、森林和二叉树的对应关系

设森林

$F = (T_1, T_2, \cdots, T_n)$;

$T_1 = (root, t_{11}, t_{12}, \cdots, t_{1m})$;

二叉树

B =(LBT, Node(root), RBT);

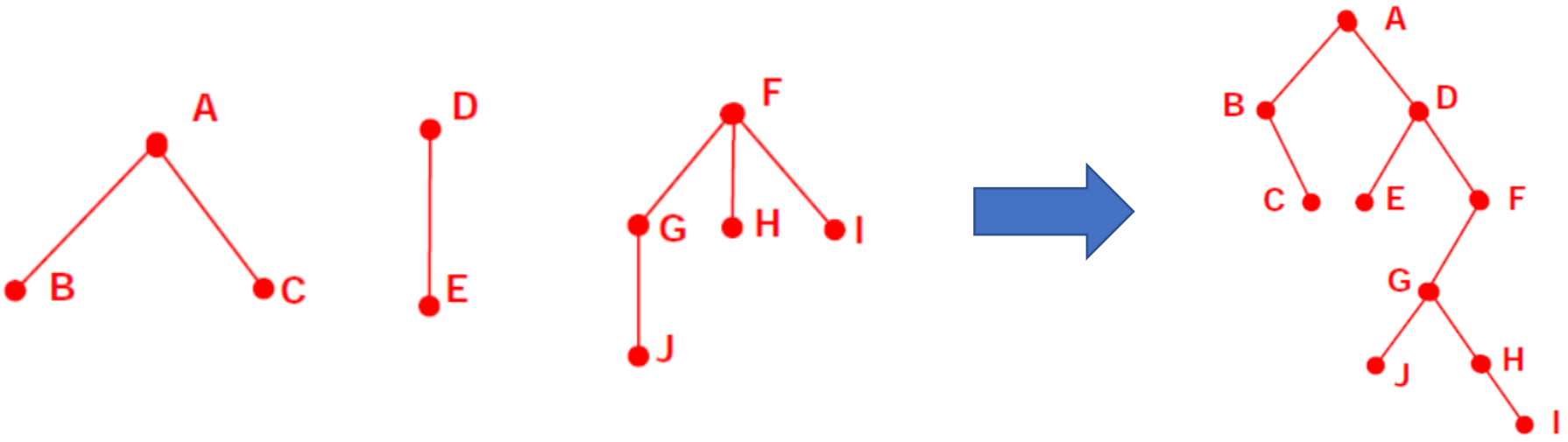
由森林转换成二叉树的转换规则为：

若 **F = Φ**，则 **B = Φ**；

否则，由 **ROOT(T_1)** 对应得到 **Node(root)**；

由 **(t_{11}, t_{12}, …, t_{1m})** 对应得到 **LBT**；

由 **(T_2, T_3,…, T_n)** 对应得到 **RBT**。



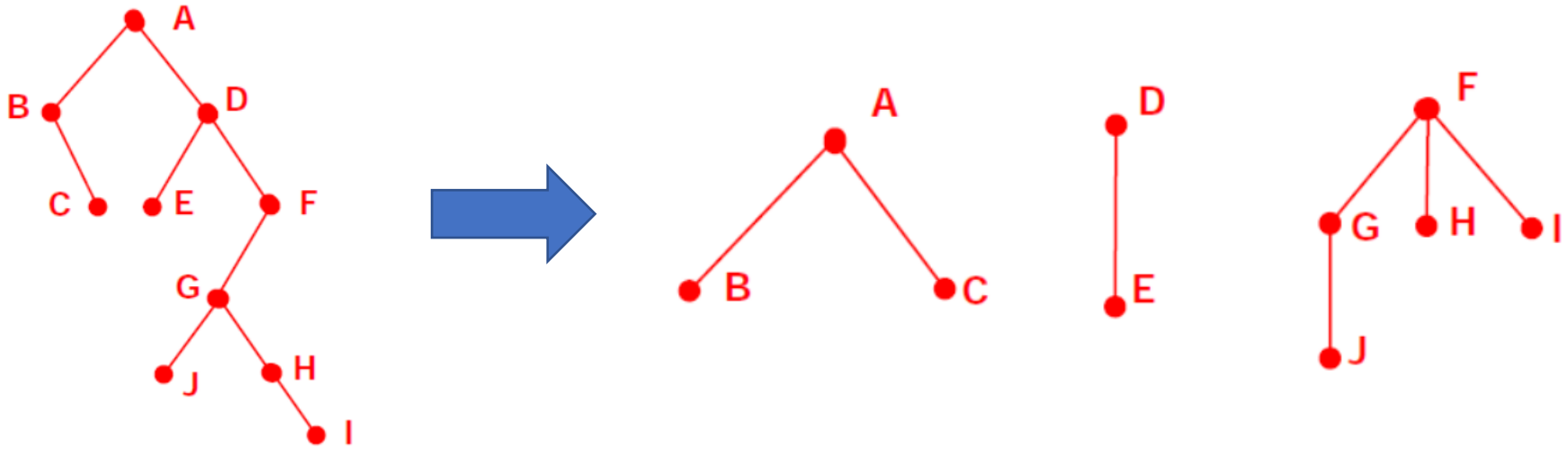
由二叉树转换为森林的转换规则为：

若 **B = Φ**， 则 **F = Φ**；

否则，由 **Node(root)** 对应得到 **ROOT(T_1)**；

由**LBT** 对应得到 **(t_{11}, t_{12}, …， t_{1m})**；

由**RBT** 对应得到 **(T_2, T_3, …, T_n)**。



6.7　树和森林的遍历

一、树的遍历

二、森林的遍历

一、树的遍历　有三条搜索路径:

先根(次序)遍历:

若树不空，则先访问根结点，然后依次先根遍历各棵子树。

后根(次序)遍历:

若树不空，则先依次后根遍历各棵子树，然后访问根结点。

按层次遍历:

若树不空，则自上而下自左至右访问树中每个结点。

先根遍历时顶点的访问次序：

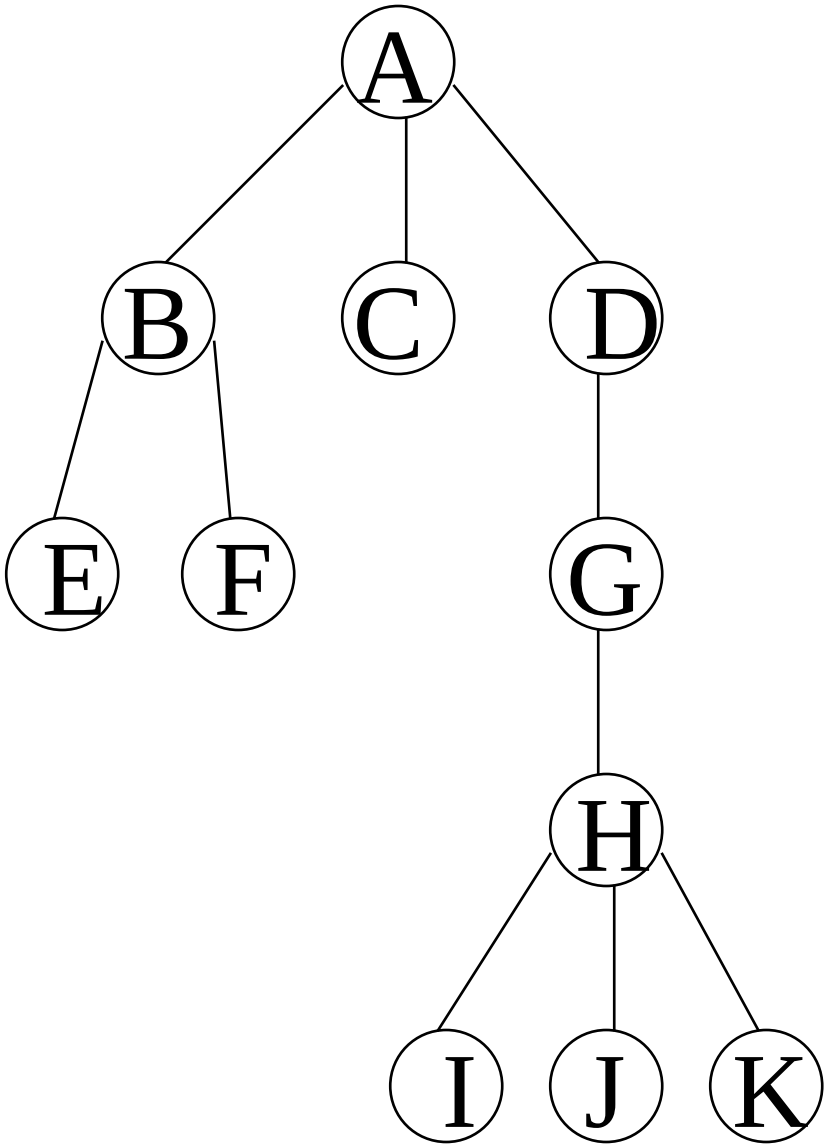
A B E F C D G H I J K

后根遍历时顶点的访问次序：

E F B C I J K H G D A

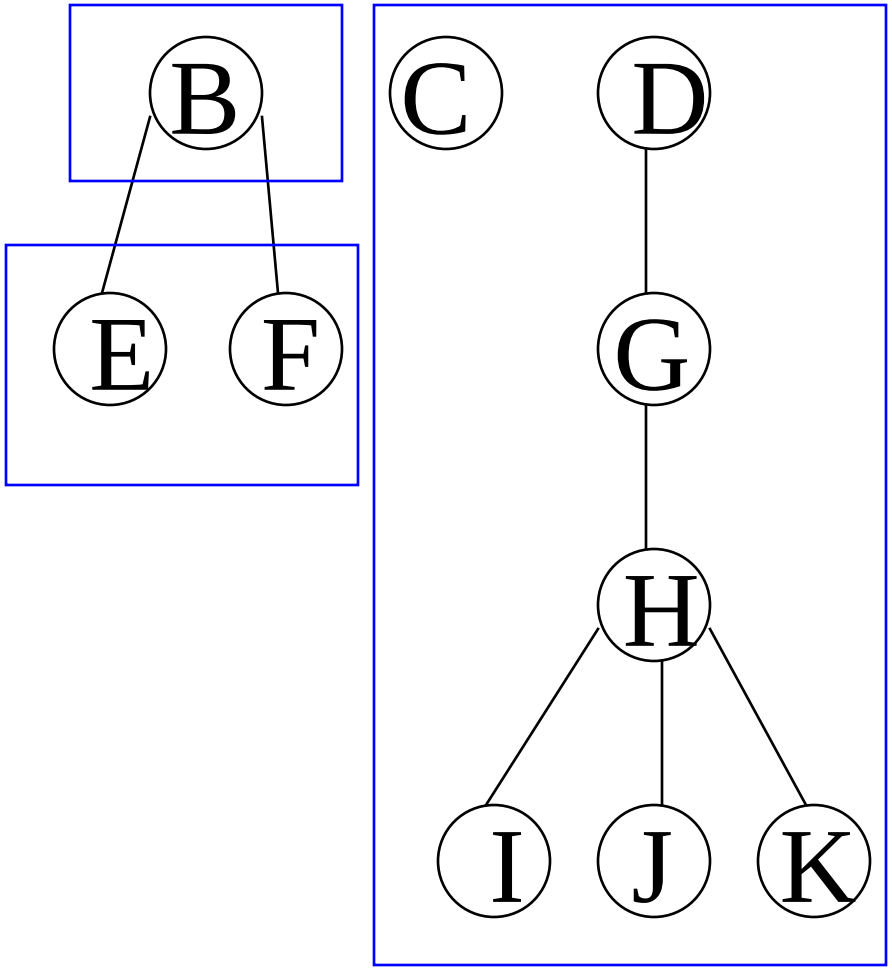
层次遍历时顶点的访问次序：

A B C D E F G H I J K



二、森林的遍历

森林由三部分构成：



1．森林中第一棵树的根结点；

2．森林中第一棵树的子树森林；

3．森林中其它树构成的森林。

1. 先序遍历

若森林不空，则

访问森林中第一棵树的根结点；

先序遍历森林中第一棵树的子树森林；

先序遍历森林中(除第一棵树之外)其余树构成的森林。

即：**依次从左至右**对森林中的每一棵**树**进行**先根遍历**。

2．中序遍历

若森林不空，则

中序遍历森林中第一棵树的子树森林；

访问森林中第一棵树的根结点；

中序遍历森林中(除第一棵树之外)其余树构成的森林。

即：**依次从左至右**对森林中的每一棵**树**进行**后根遍历**。

森林、树和二叉树遍历的对应关系？

森林		树		二叉树
先序遍历	➡	先根遍历	➡	先序遍历
中序遍历	➡	后根遍历	➡	中序遍历

6.8 哈夫曼树与哈夫曼编码

一. 最优树的定义

二. 如何构造最优树

三. 前缀编码

一、最优树的定义

- **结点的路径长度**定义为：

  从根结点到该结点的路径上分支的数目。

- **树的路径长度**定义为：

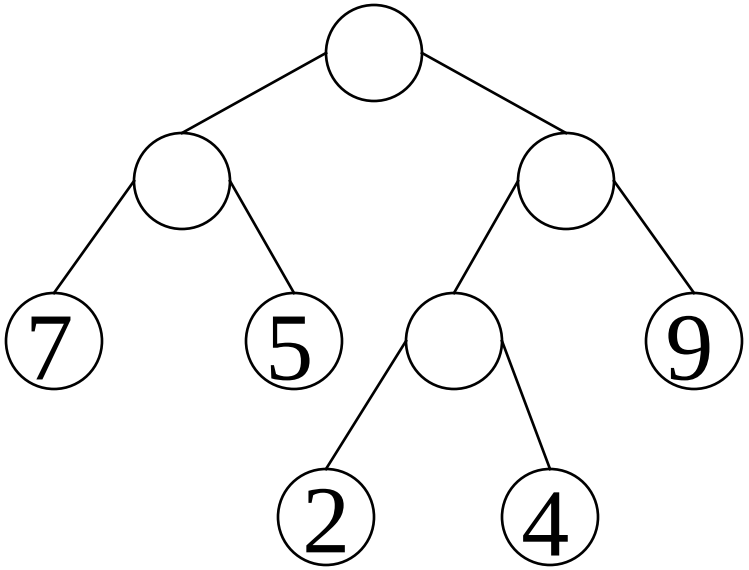
  树中每个结点的路径长度之和。

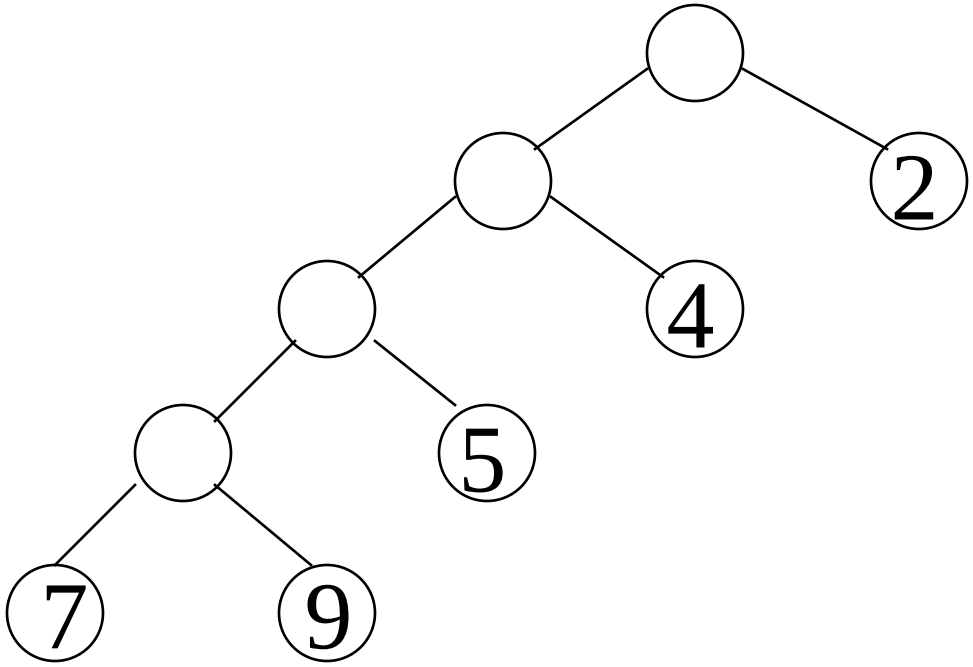
- **树的带权路径长度**定义为：

  树中所有**叶子结点的带权路径长度**之和

  $WPL(T) = \Sigma w_k l_k$ (对所有叶子结点)。

在所有含 *n* 个叶子结点、并带相同权值的 m 叉树中，必存在一棵其**带权路径长度取最小值**的树，称为“**最优树**”。





WPL(T)=

7×2+5×2+2×3+

4×3+9×2

=60

WPL(T)=

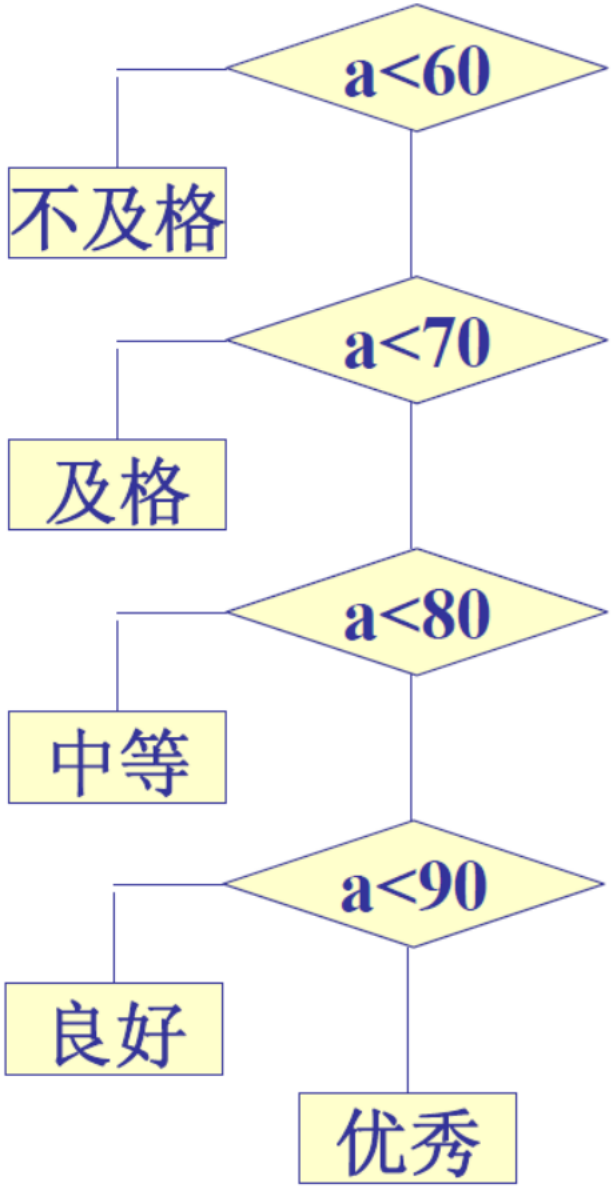
7×4+9×4+5×3+

4×2+2×1

=89

在解某判定问题时，利用哈夫曼树可以得到最佳判定算法。

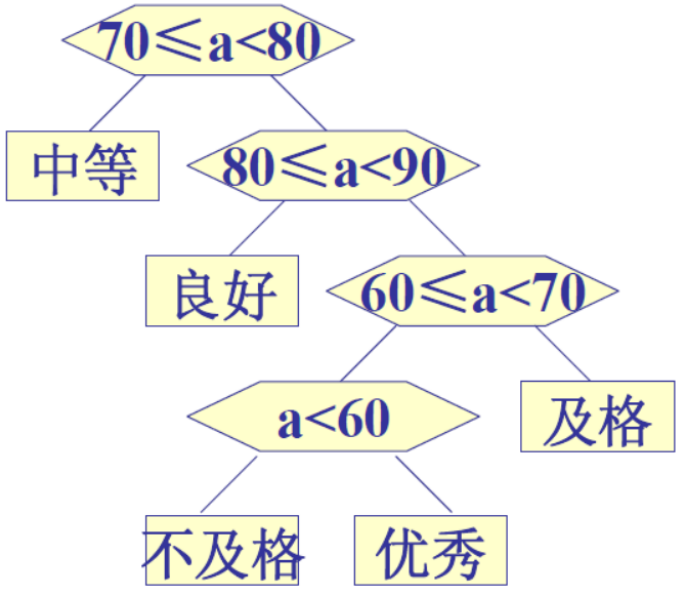
例：编程将百分制转换成五级分制。判定过程所用判定树：



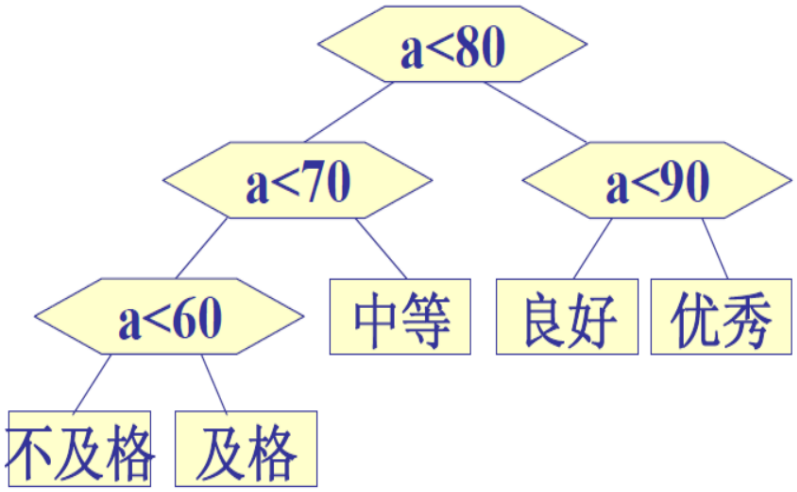
如果上述程序需反复使用，且每次的输入量很大，则应考虑上述程序的质量问题，即其操作所需的时间。因为在实际生活中，学生的成绩在五个等级上的分布是不均匀的，假设其分布规律如下表所示，则80%以上的数据需进行三次或三次以上的比较才能得出结果。

分数	0~59	60~69	70~79	80~89	90~100
比例	0.05	0.15	0.40	0.30	0.10

以学生成绩在五个等级上的分布概率为权，构造具有五个叶子结点的哈夫曼树，得图示判定过程，可使大部分数据经过较少的比较次数得出结果。



将每个判定框的两次比较分开，得到左图的判定树。



二、如何构造最优树

赫夫曼算法:

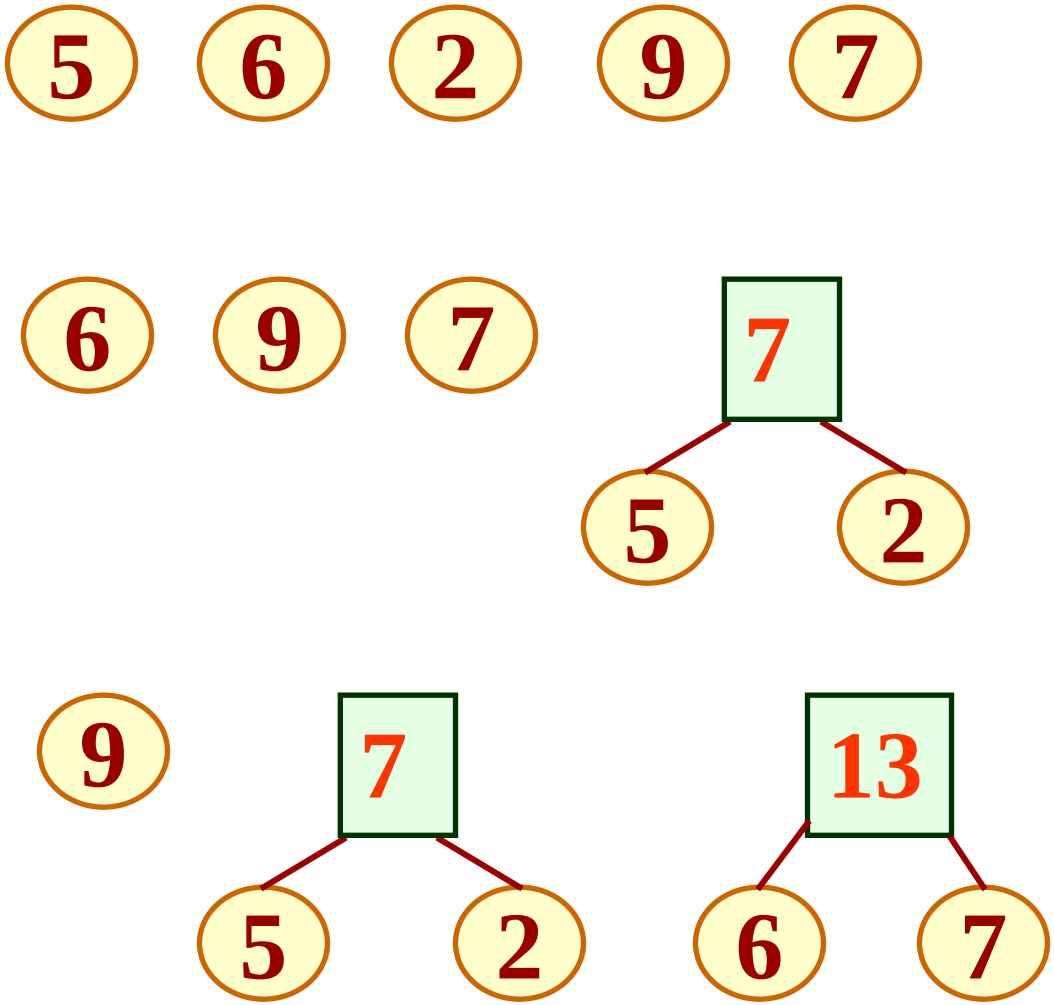
① 根据给定的n个权值{w1, w2, …, w3}构成n棵二叉树的集合F={T1, T2, …, Tn}，其中每棵二叉树Ti中只有一个带权为wi的根结点，其左右子树均空；

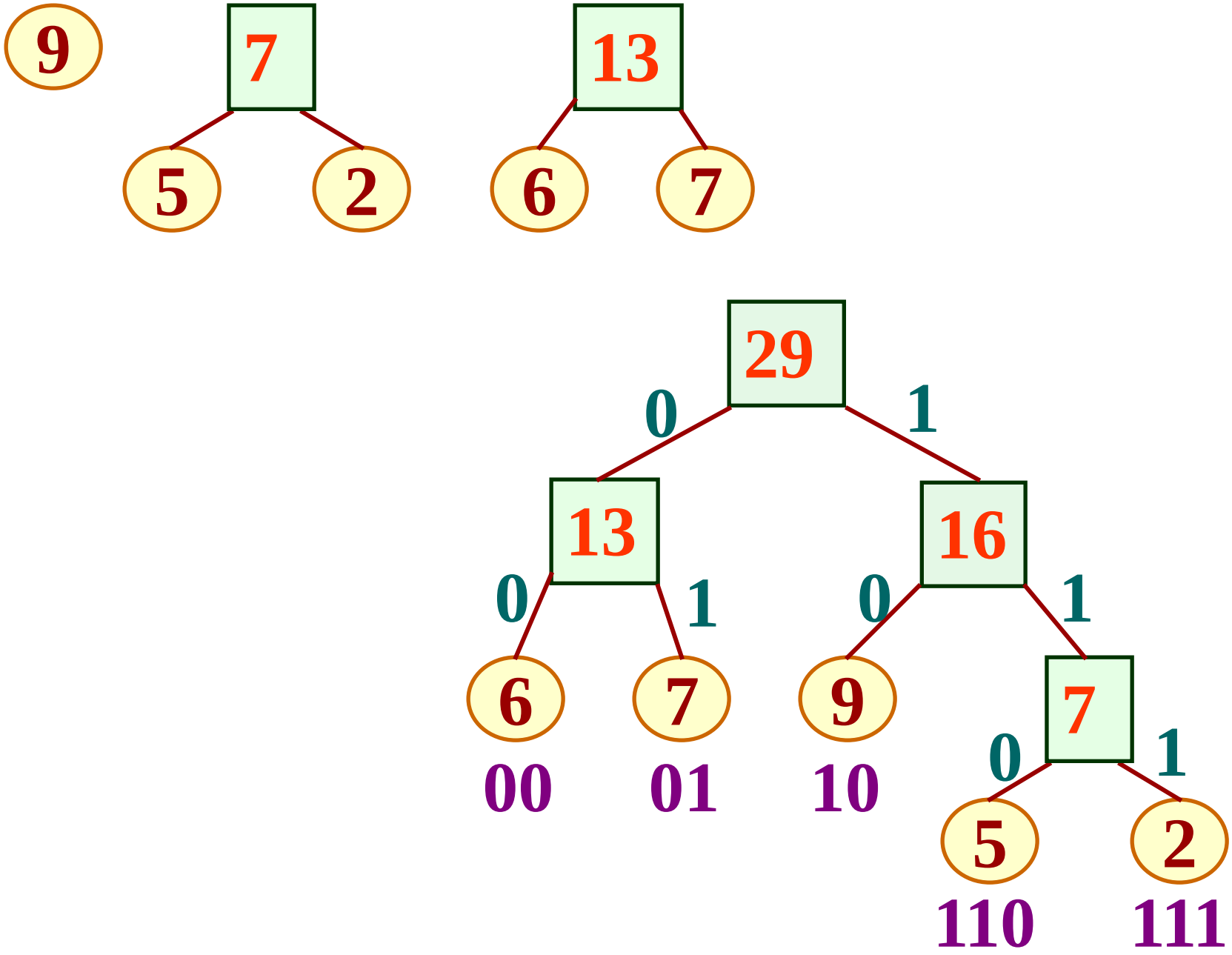
② 在F中选取两棵根结点的权值最小的树作为左、右子树构造一棵新的二叉树，且置新的二叉树的根结点的权值为其左、右子树上根结点的权值之和；

③ 在F中删除这两棵树，将新得到的二叉树加入F中；

④ 重复②和③，直到F只含一棵树为止。这棵树便是哈夫曼树。

例如：已知权值 W={ 5, 6, 2, 9, 7 }




9
7
5
2
13
6
7
29
0
1
13
16
0
1
0
1
6
7
9
7
0
1
00
01
10
5
2
110
111

三、前缀编码

电文：ABACCDA

A:00
B:01
C:10
D:11

00 01 00 10 10 11 00
A B A C C D A

A:0
B:00
C:1
D:01

0 0 0 0 1 1 0 1 0
AAAA
B B
A B A

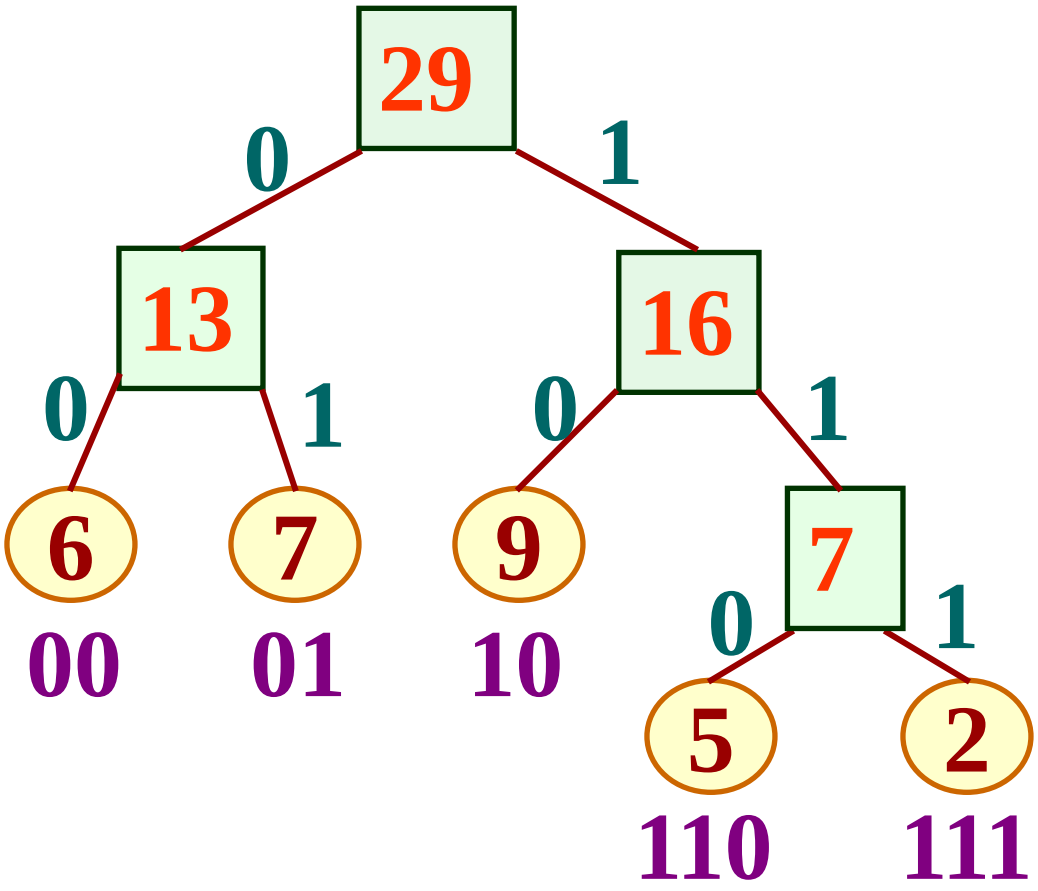
前缀编码指的是，任何一个字符的编码都不是同一字符集中另一个字符的编码的前缀。

利用赫夫曼树可以构造一种不等长的二进制编码，并且构造所得的赫夫曼编码是一种**最优前缀编码**，即使所传电文的总长度最短。

如何构造哈夫曼编码？

构造方法：

设计电文总长最短的二进制前缀编码即为以n种字符出现的频率作权，设计一棵哈夫曼树的问题，约定左分支表示字符0，右分支表示字符1，则从根结点到叶结点的路径上，分支字符组成的字符串作为该叶子结点字符的编码，由此得到的二进制前缀编码称为**哈夫曼编码**。


29
0
1
13
16
0
1
0
1
6
7
9
7
00
01
10
0
1
5
2
110
111

本章小结

① 熟练掌握二叉树的结构特性，了解相应的证明方法。

② 熟悉二叉树的各种存储结构的特点及适用范围。

③ 遍历二叉树是二叉树各种操作的基础。实现二叉树遍历的具体算法与所采用的存储结构有关。掌握各种遍历策略的递归算法，灵活运用遍历算法实现二叉树的其它操作。层次遍历是按另一种搜索策略进行的遍历。

④ 理解二叉树线索化的实质是建立结点与其在相应序列中的前驱或后继之间的直接联系，熟练掌握二叉树的线索化过程以及在中序线索化树上找给定结点的前驱和后继的方法。二叉树的线索化过程是基于对二叉树进行遍历，而线索二叉树上的线索又为相应的遍历提供了方便。

⑤ 熟悉树的各种存储结构及其特点，掌握树和森林与二叉树的转换方法。

⑥ 了解最优树的特性，掌握建立最优树和哈夫曼编码的方法。